

Colegio Universitario de Cartago
Escuela de Tecnologías de Información

Exploración práctica y comparativa de mecanismos internos de sistemas operativos modernos: el caso de Windows Server 2025

Steven, Fabricio y Vincent
Colegio Universitario de Cartago
TI-163 Arquitectura y Sistemas Operativos
Ing. Greivin Vásquez Valverde
Programada 2 — II Cuatrimestre 2026

Índice

Contenido

Índice	2
1. Introducción	4
2. Instalación del Sistema Operativo en Máquina Virtual	4
<i>Paso 1. Instalación de Oracle VirtualBox</i>	4
<i>Paso 2. Descarga de la imagen ISO de Windows Server 2025</i>	5
<i>Paso 3. Creación de la máquina virtual</i>	6
<i>Paso 4. Instalación de Windows Server dentro de la VM</i>	6
<i>Paso 5. Instalación de Guest Additions</i>	10
Distribución de trabajo del grupo	14
3. Componentes del Sistema Operativo	14
3.1 Gestión de Memoria Virtual y Paginación (Steven)	14
3.1.1 <i>Marco teórico</i>	15
3.1.2 <i>Herramientas de monitoreo utilizadas</i>	15
3.1.3 <i>Metodología del experimento</i>	16
3.1.4 <i>Evidencia 1 — Configuración del archivo de paginación</i>	17
3.1.5 <i>Evidencia 2 — Prueba de reserva de memoria de 500 MB</i>	19
3.1.6 <i>Evidencia 3 — Prueba de reserva de memoria de 1200 MB con Monitor de Rendimiento</i>	21
<i>Prueba — antes</i>	22
<i>Prueba — durante</i>	23
<i>Prueba — después</i>	24
3.1.7 <i>Preguntas de la rúbrica — Windows Server</i>	24
3.2 Sockets (comunicación entre procesos en red)	25
3.3 Tuberías (pipes y pipes con nombre)	25
3.4 Mensajes entre procesos (IPC basado en colas o memoria compartida)	26
3.5 Creación de procesos e hilos (Fork o su equivalente)	26
3.6 Semáforos y Mutex	26
4. Comparación con Otro Sistema Operativo	27
5. Código Fuente	27
6. Conclusiones	31
Referencias.....	32

Anexos	32
---------------------	-----------

1. Introducción

Windows Server es la plataforma de servidor empresarial de Microsoft que permite a las organizaciones ejecutar y proteger aplicaciones, servicios y cargas de trabajo en entornos locales, híbridos y en la nube.

Tanto si se trata de una pequeña empresa como de una infraestructura empresarial compleja, Windows Server proporciona la base para la computación segura, escalable y de alto rendimiento que crece junto con la organización. Apoyado en décadas de innovación del ecosistema Windows, sirve como columna vertebral de millones de organizaciones a nivel mundial, sosteniendo desde servidores de archivos y aplicaciones web hasta cargas de trabajo empresariales complejas y soluciones impulsadas por inteligencia artificial.

El presente informe documenta, mediante código y evidencia práctica, el funcionamiento interno de los componentes clave de Windows Server 2025 instalado en una máquina virtual: gestión de memoria virtual y paginación, sockets, tuberías, mensajería entre procesos, creación de procesos e hilos, y mecanismos de sincronización (semáforos y mutex). Asimismo, se compara el comportamiento de estos mecanismos con los de otro sistema operativo, en este caso Linux.

2. Instalación del Sistema Operativo en Máquina Virtual

A continuación se documenta el proceso seguido para instalar Windows Server 2025 (edición en español) sobre Oracle VirtualBox, utilizado como entorno de pruebas para todo el resto del informe.

Paso 1. Instalación de Oracle VirtualBox

Se descargó VirtualBox desde el sitio oficial (<https://www.virtualbox.org/wiki/Downloads>), eligiendo la versión para Windows Host, y se instaló dejando la configuración por defecto durante el asistente.

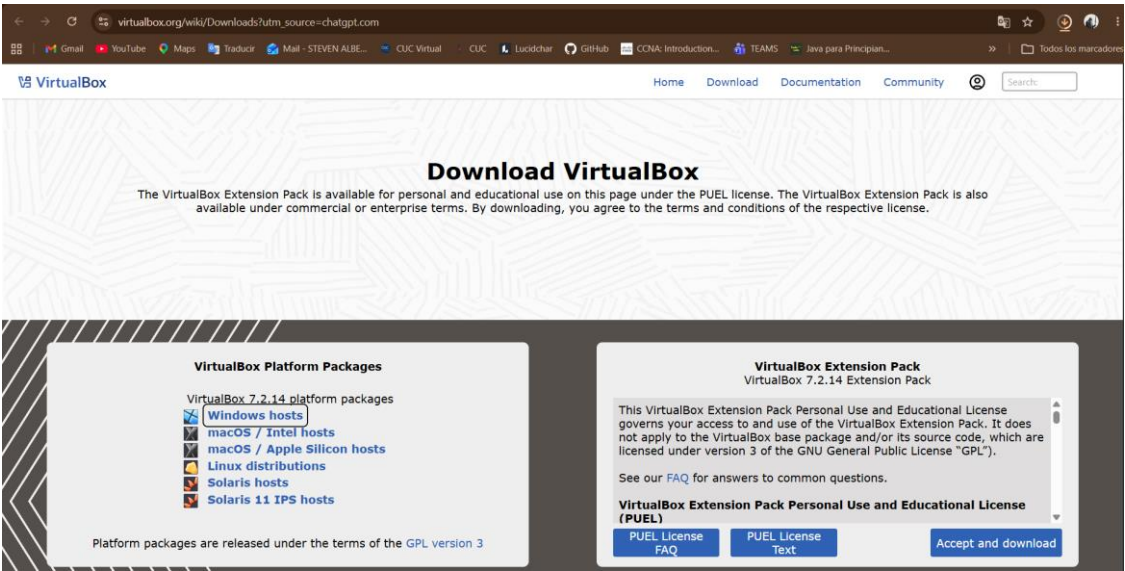


Figura 1. Página oficial de descarga de VirtualBox.

Paso 2. Descarga de la imagen ISO de Windows Server 2025

Se descargó la ISO de evaluación de Windows Server 2025, en su versión en español, desde el Centro de Evaluación de Microsoft (<https://www.microsoft.com/es-es/evalcenter/download-windows-server-2025>).

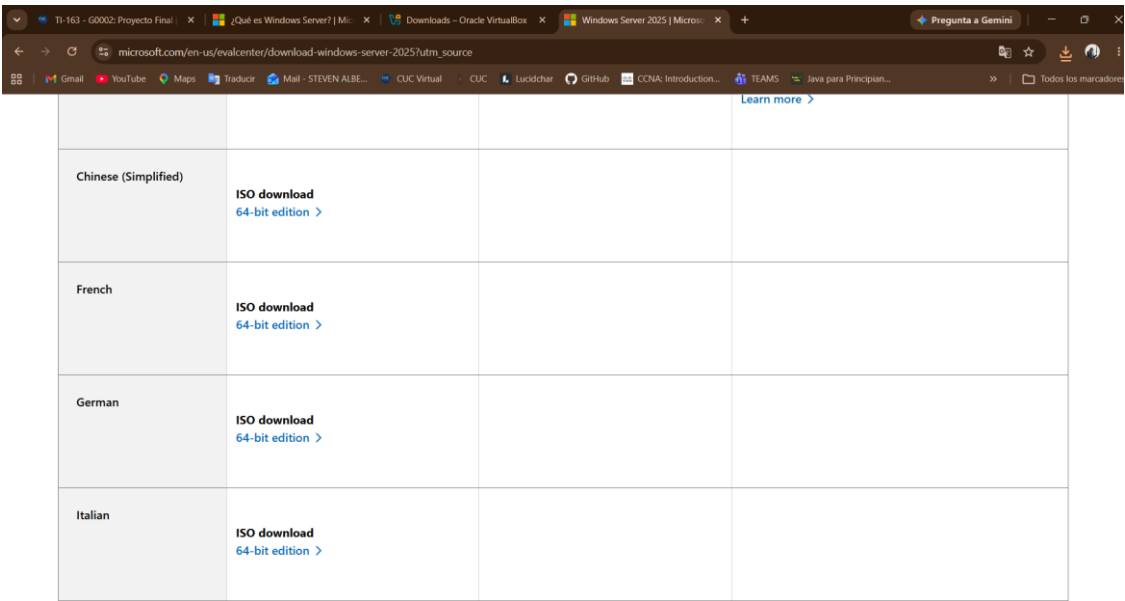


Figura 2. Centro de evaluación de Microsoft para Windows Server 2025.

Paso 3. Creación de la máquina virtual

Dentro de VirtualBox se creó una nueva máquina virtual y se seleccionó la ISO descargada, sin marcar la opción "Proceed with Unattended Installation", ya que el proceso de instalación se realizó manualmente.

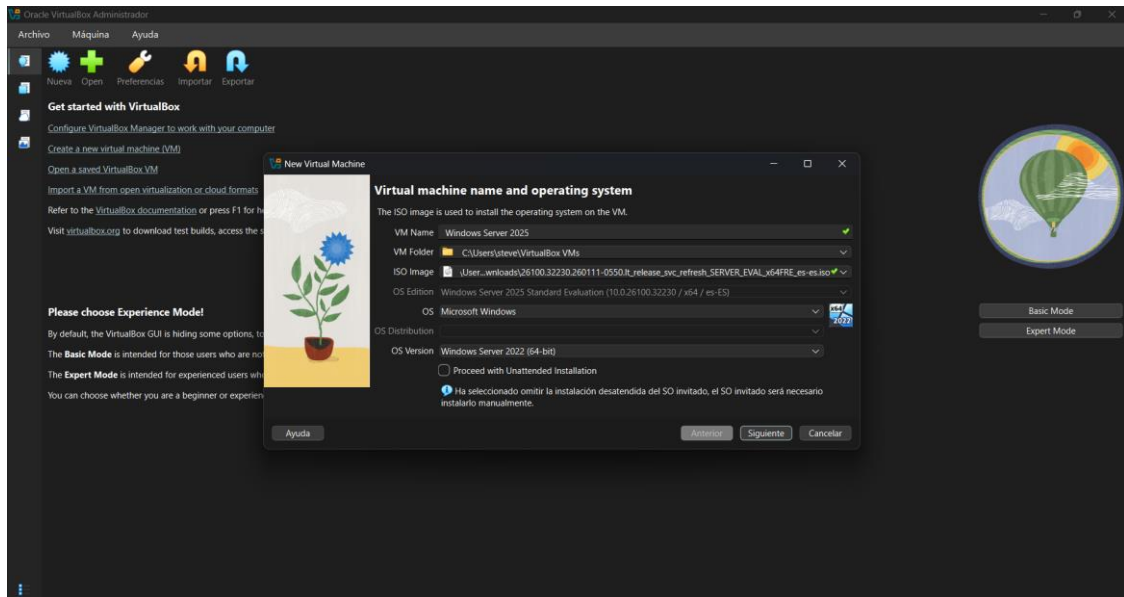


Figura 3. Selección de la ISO al crear la máquina virtual.

Se asignaron los recursos de hardware (memoria RAM, CPU y disco) a la máquina virtual creada para esta tarea.

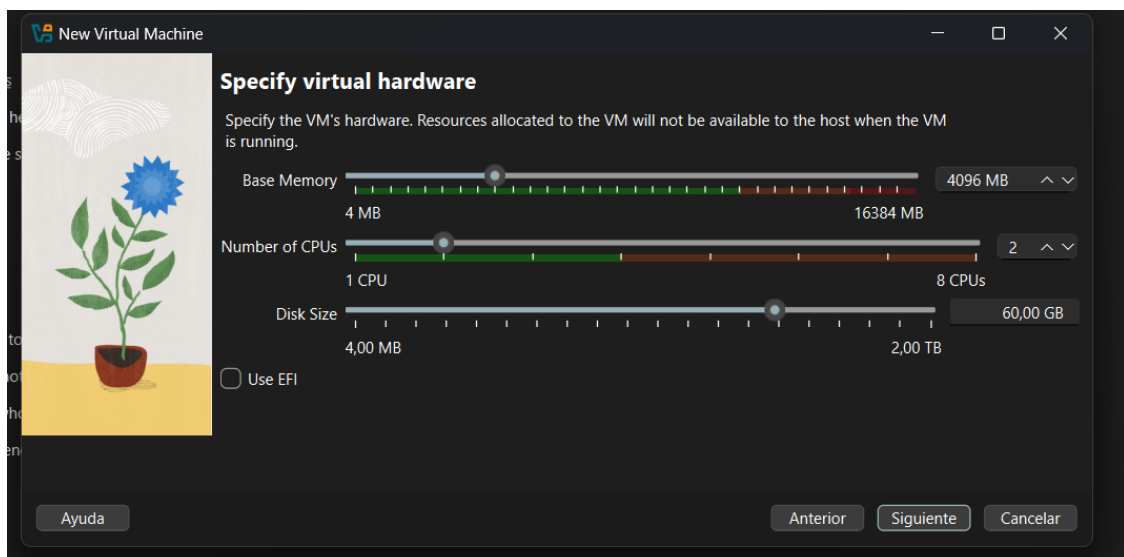


Figura 4. Asignación de recursos a la VM.

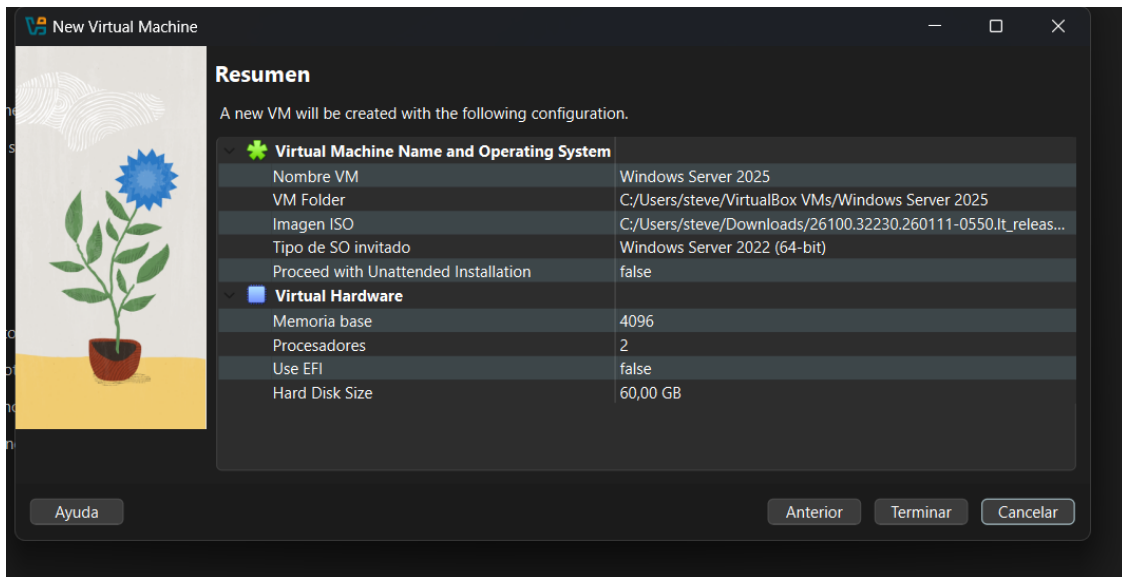


Figura 5. Resumen de configuración antes de finalizar la creación de la VM.

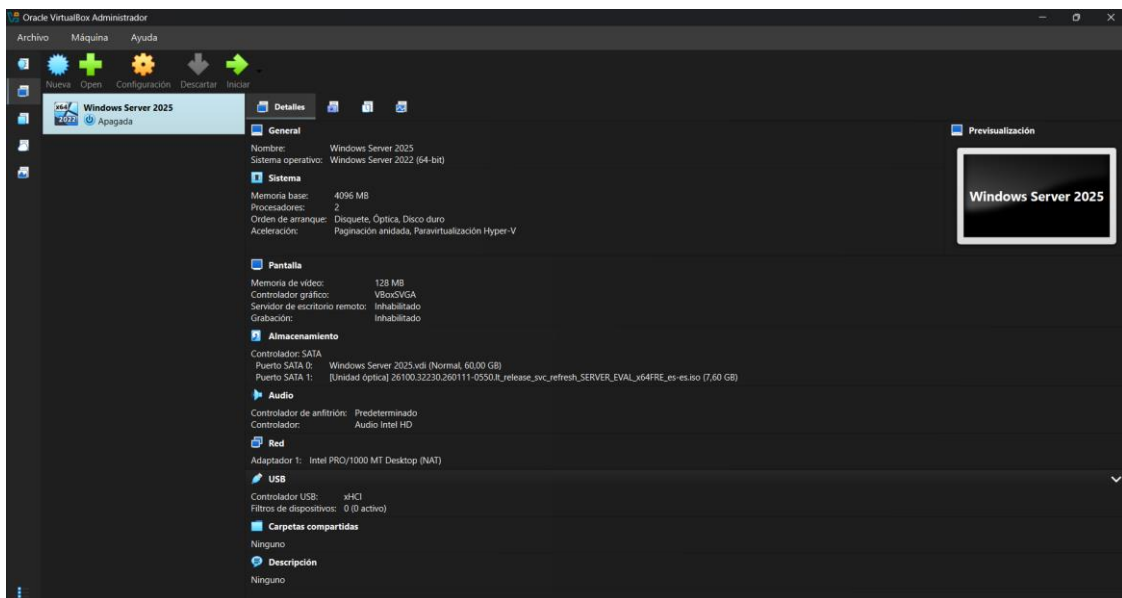
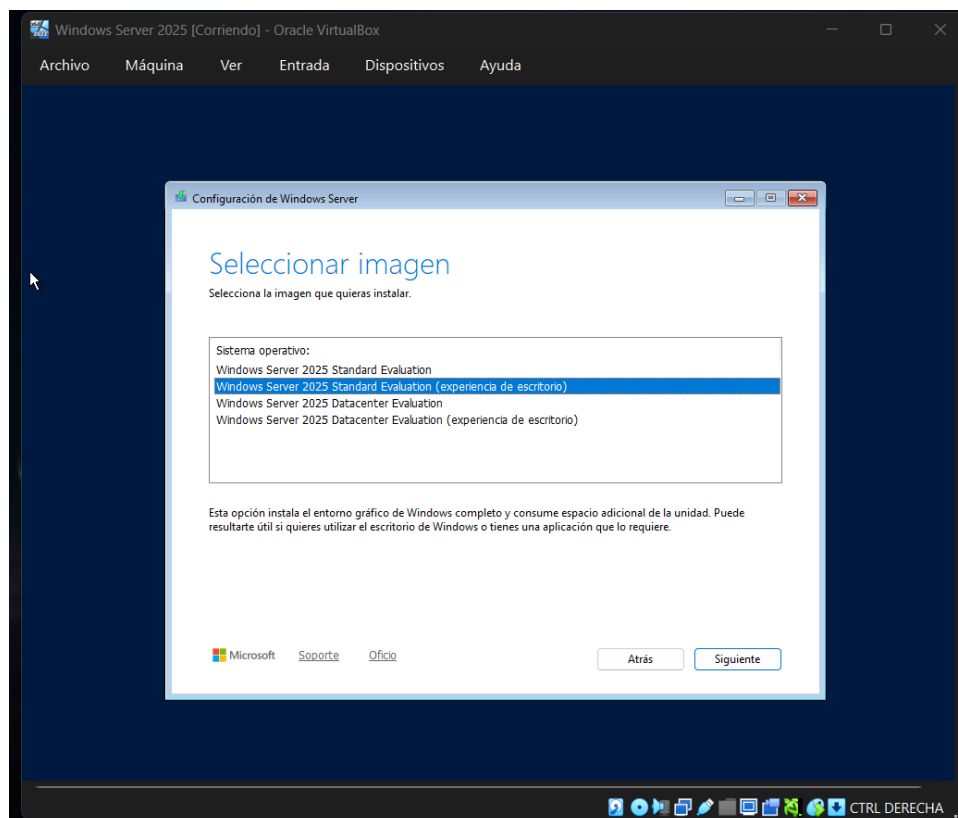
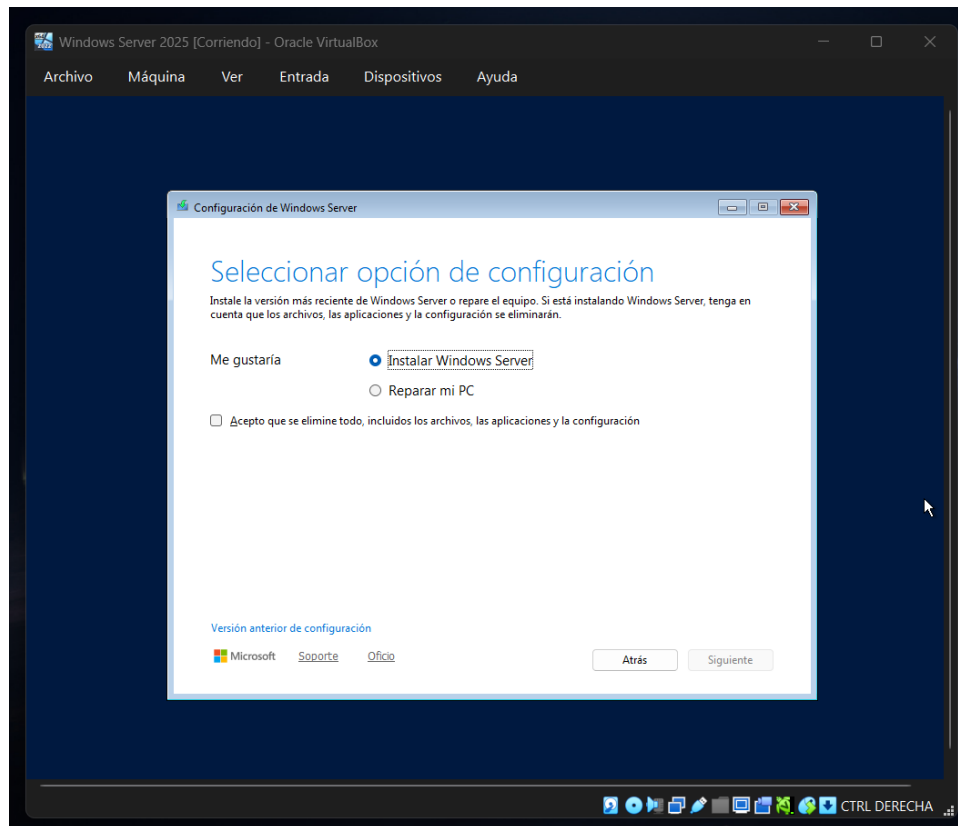


Figura 6. Máquina virtual creada correctamente.

Paso 4. Instalación de Windows Server dentro de la VM

Tras configurar hora y teclado, se seleccionó instalar Windows Server y la edición correspondiente.



Figuras 7 y 8. Selección de la edición de Windows Server a instalar.

Se aceptaron los términos de licencia y se continuó con el asistente de instalación.

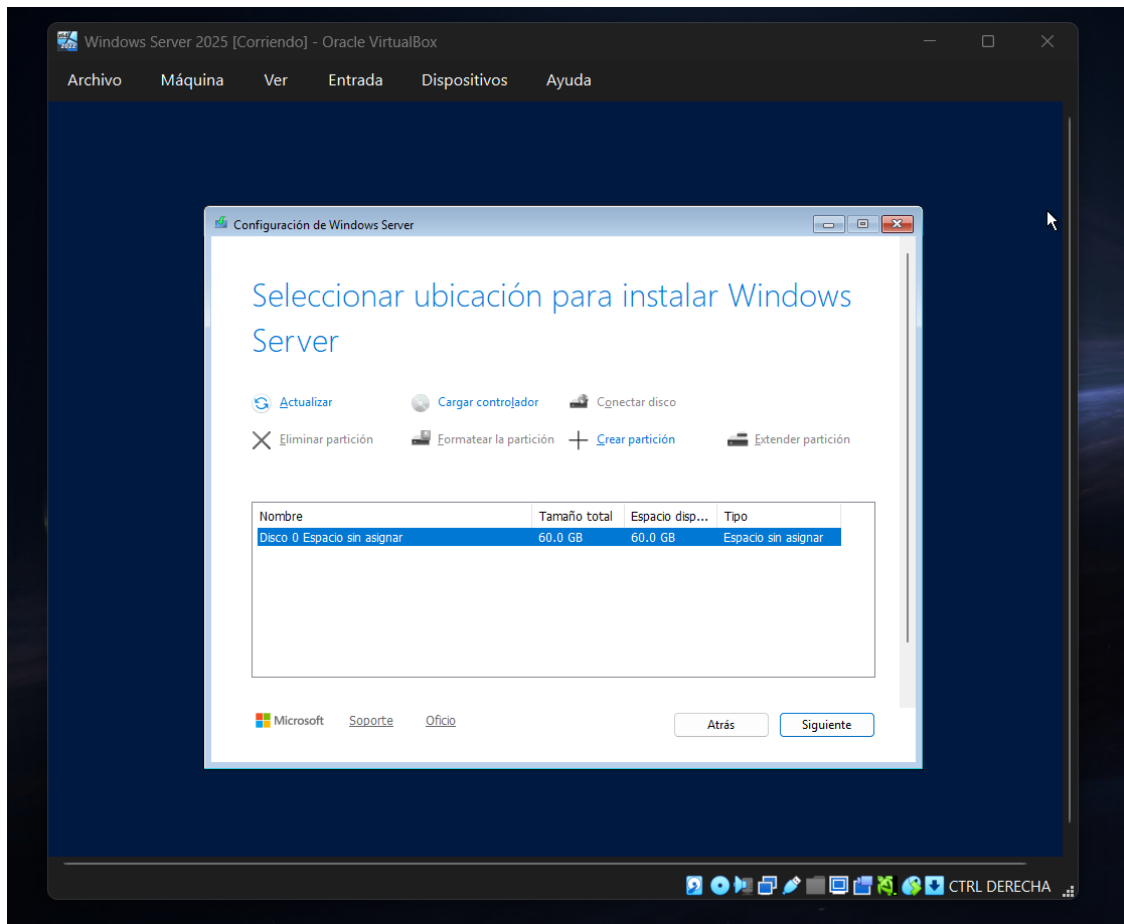


Figura 9. Aceptación de términos de licencia.

Nota importante: durante la instalación no se debe presionar ninguna tecla cuando el sistema lo solicite, ya que esto reinicia el proceso de instalación desde cero. Una vez completada la instalación se define la contraseña de administrador.

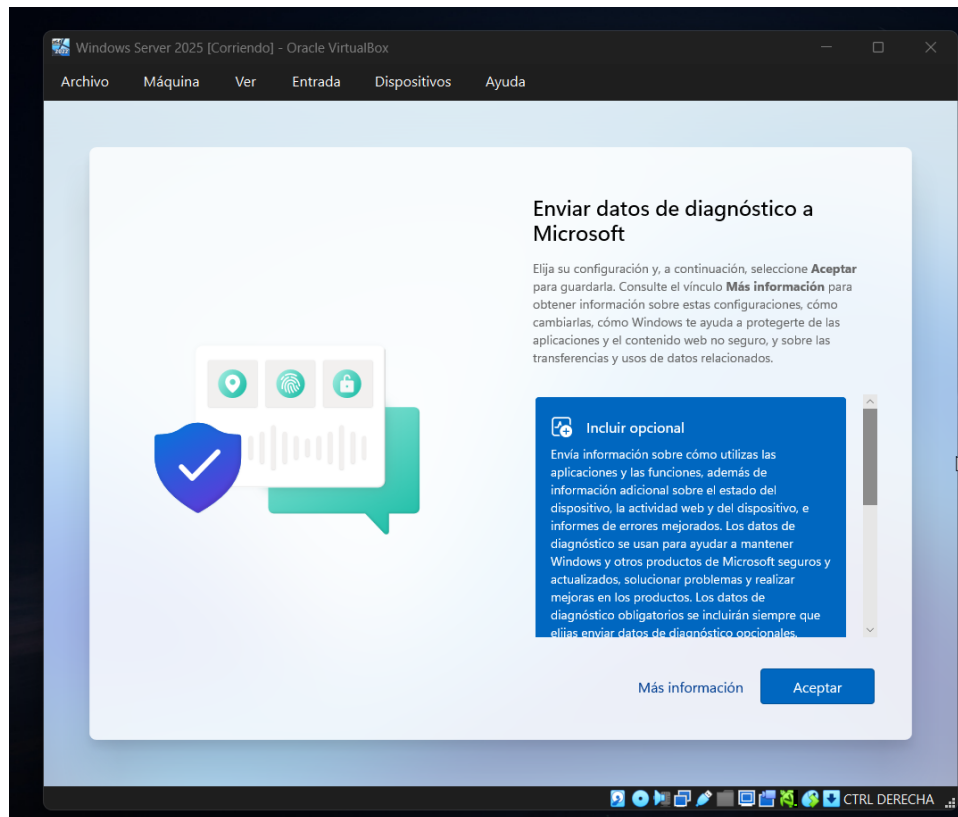


Figura 10. Definición de la contraseña de administrador.

En la configuración de privacidad se seleccionó incluir las opciones por defecto y se aceptó.

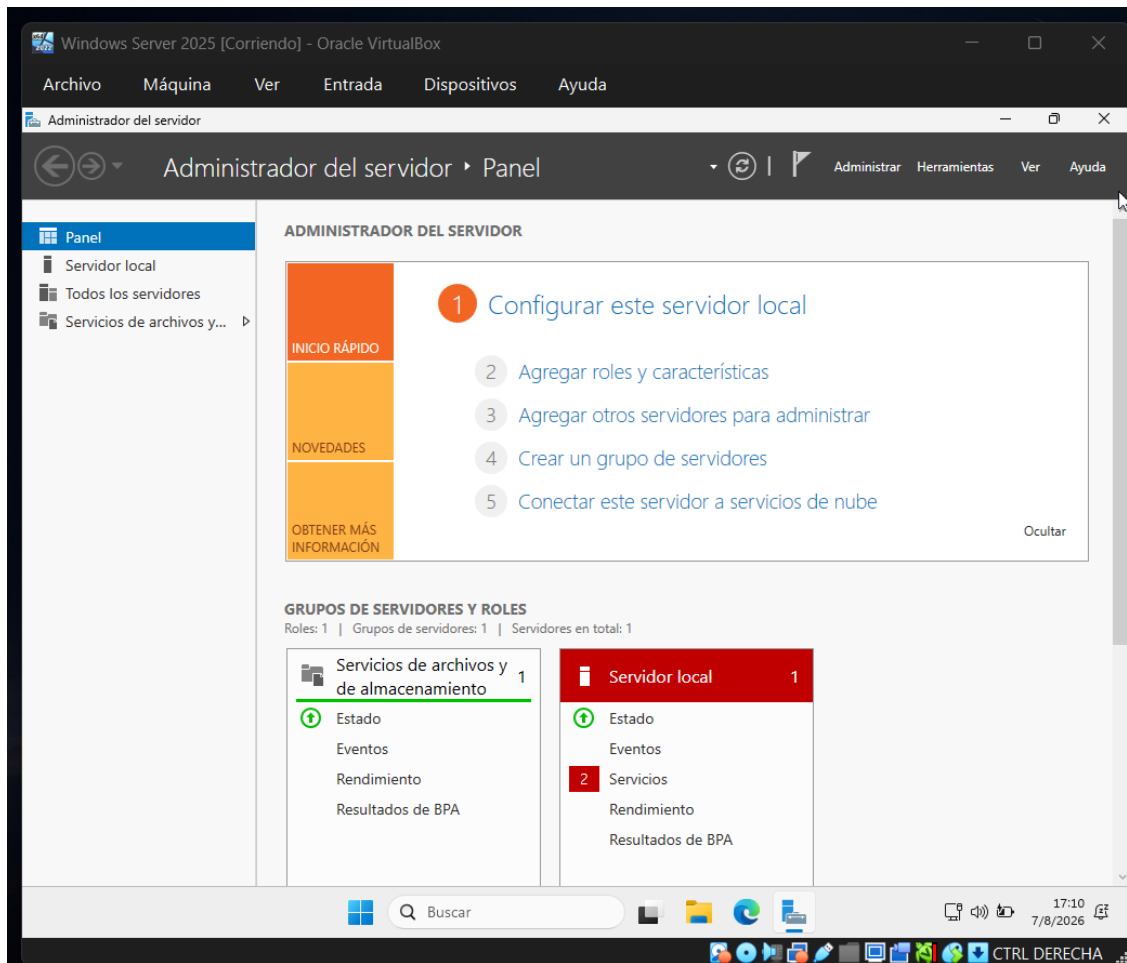


Figura 11. Configuración de privacidad al primer inicio de sesión.

Paso 5. Instalación de Guest Additions

Con Windows Server encendido, desde el menú Dispositivos → Insertar imagen de CD de las Guest Additions se habilitaron las funciones adicionales de integración entre el host y la VM.

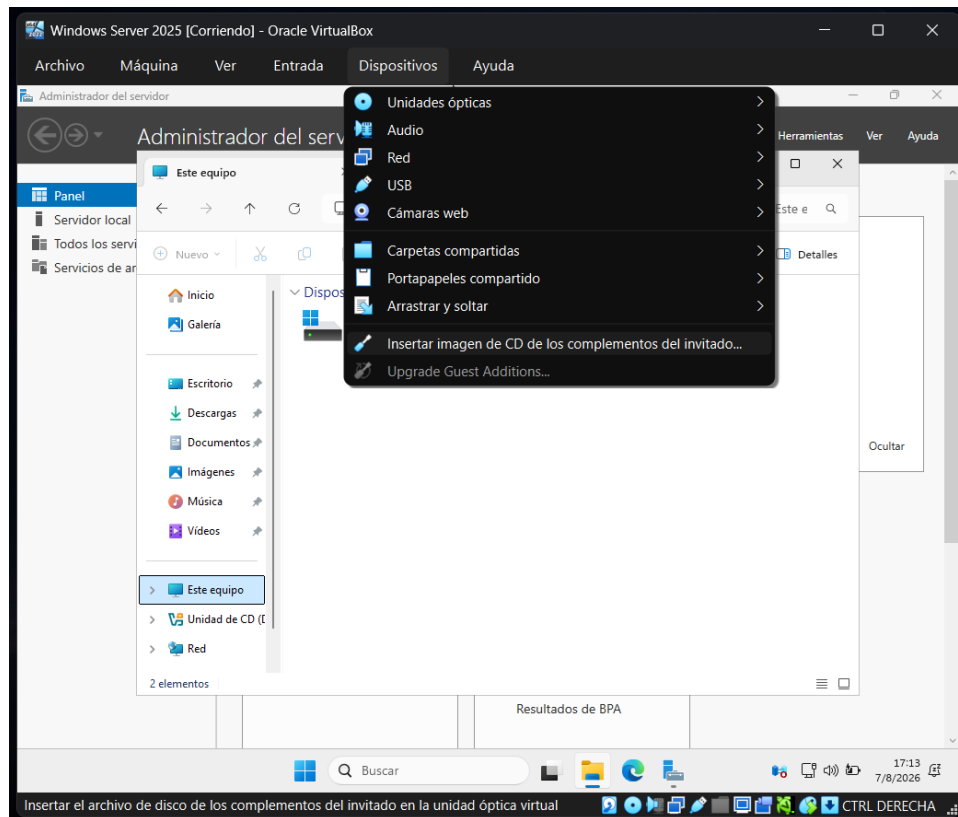


Figura 12. Inserción de la imagen de Guest Additions.

Una vez montada la imagen, la ventana de la VM se adapta automáticamente al tamaño del monitor, mejorando la fluidez de uso.

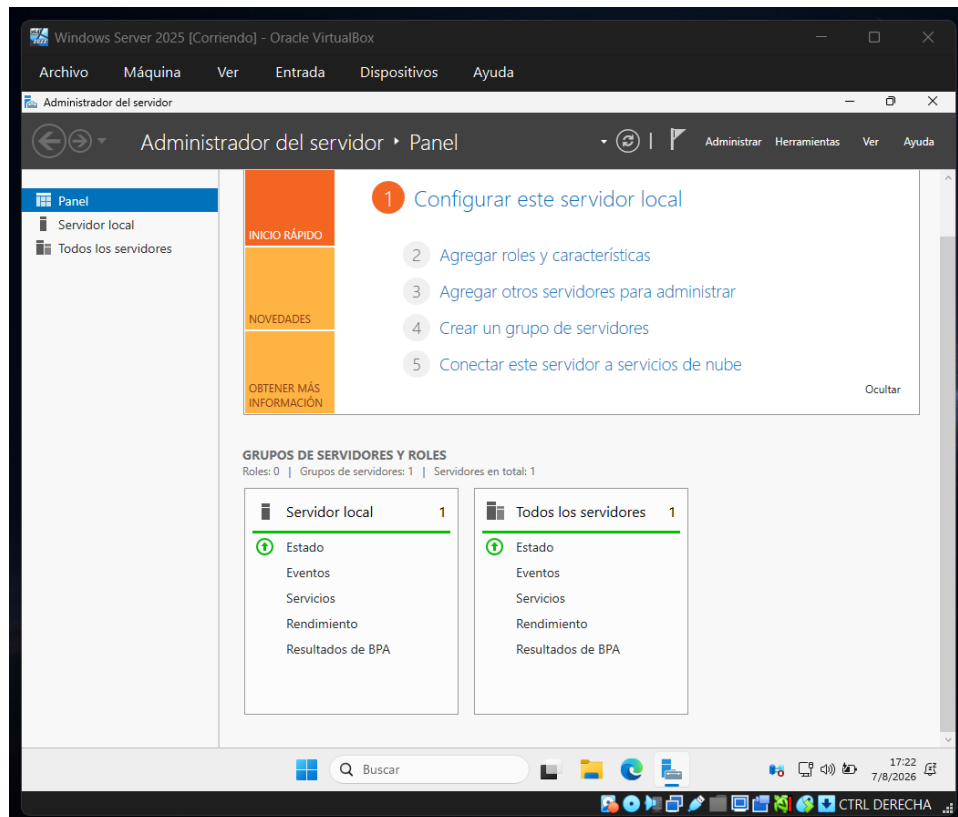


Figura 13. Ventana de la VM adaptada al monitor tras habilitar Guest Additions.

Desde "Este equipo" apareció la unidad de CD correspondiente a VirtualBox Guest Additions.

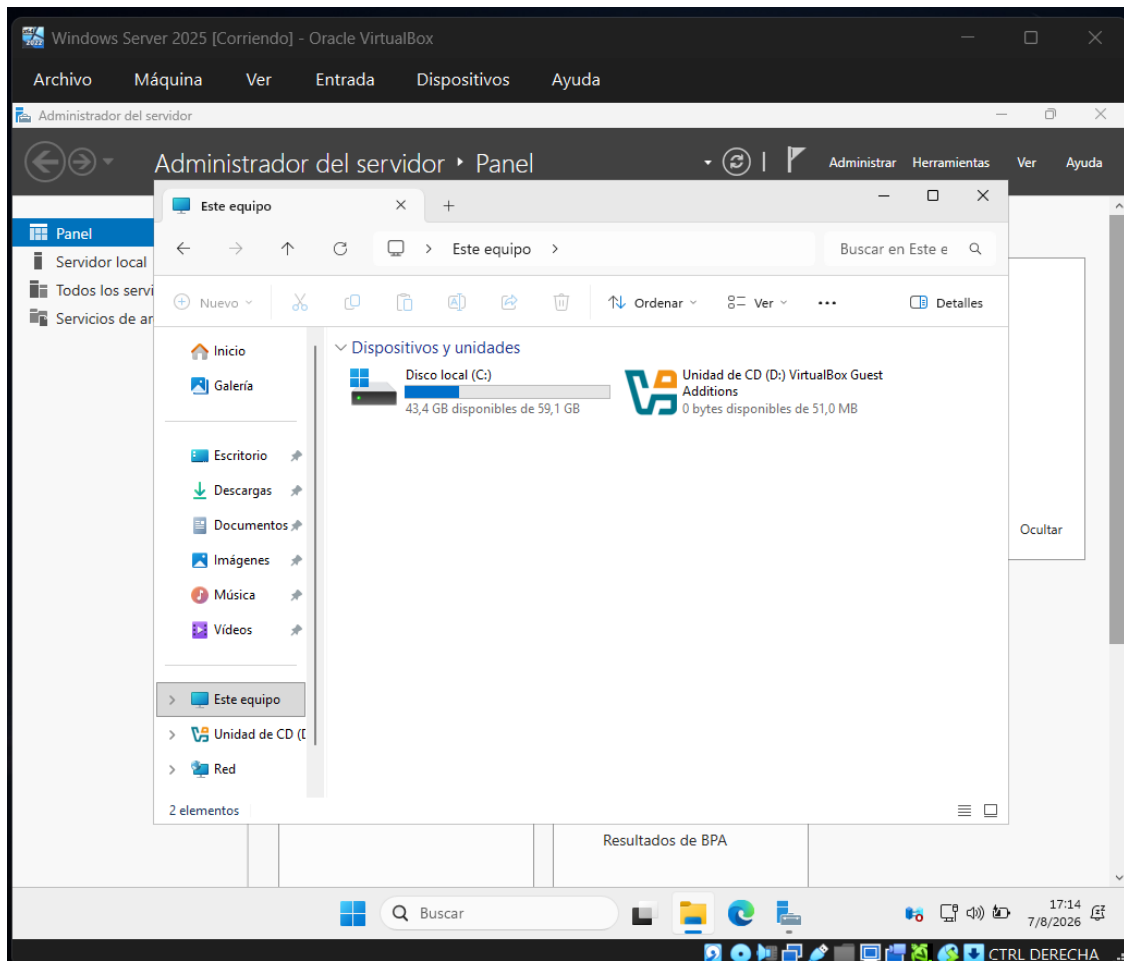


Figura 14. Unidad de CD de VirtualBox Guest Additions.

Se ejecutó el instalador dejando las opciones predeterminadas y, al finalizar, se reinició la máquina virtual.

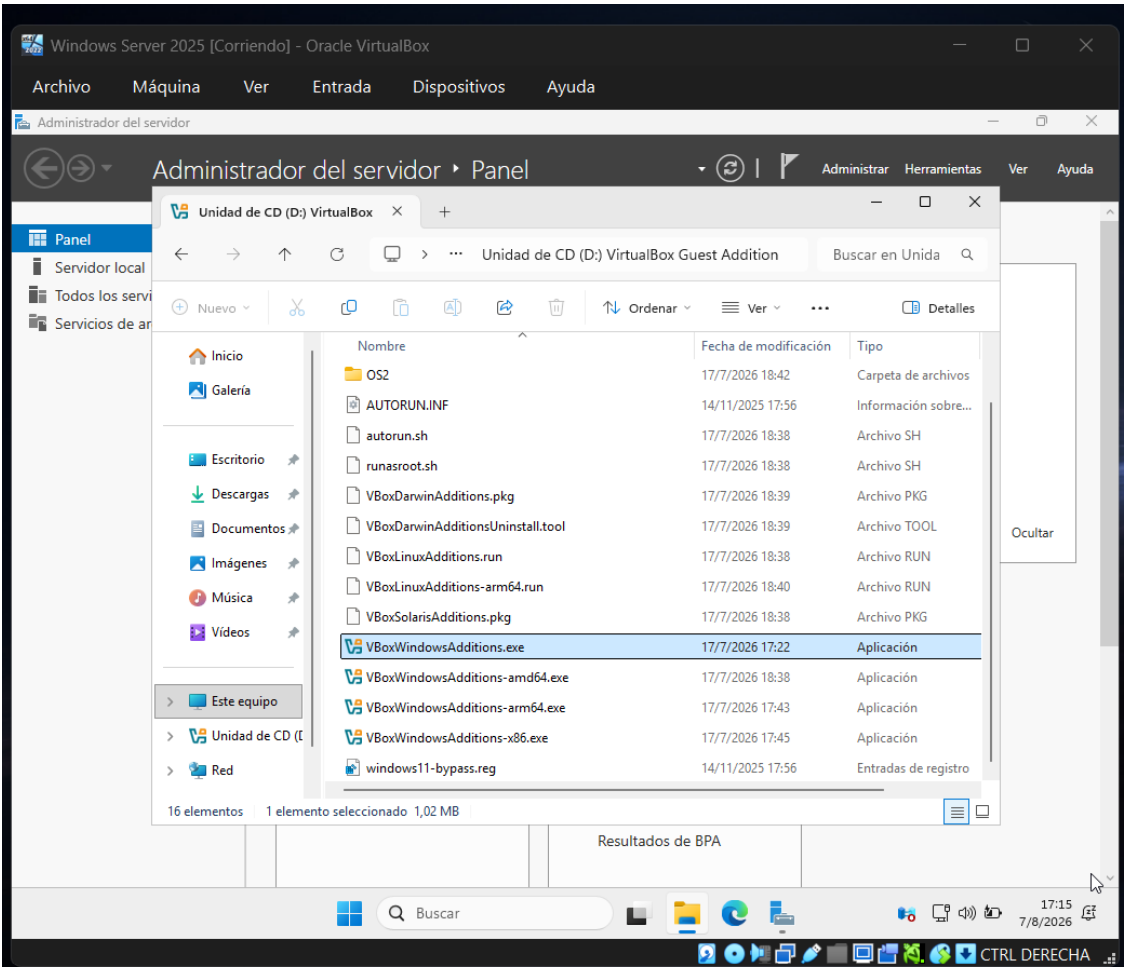


Figura 15. Instalación de Guest Additions completada.

Distribución de trabajo del grupo

El proyecto fue dividido entre los tres integrantes del grupo asignado a Windows Server, de acuerdo con los componentes solicitados en la rúbrica:

Integrante	Parte asignada	Estado
Steven	Gestión de Memoria Virtual y Paginación + Monitoreo del sistema	Completo (ver sección 3.1)
Vincent	Sockets + Pipes + IPC (colas/memoria compartida)	Completo (ver secciones 3.2, 3.3, 3.4)
Fabricio	Procesos + Hilos + Semáforos + Mutex (Fork o equivalente)	Pendiente

3. Componentes del Sistema Operativo

3.1 Gestión de Memoria Virtual y Paginación (Steven)

3.1.1 Marco teórico

En Windows, los programas trabajan normalmente con direcciones de memoria virtual y no directamente con direcciones físicas de RAM. Windows y el hardware se encargan de traducir las direcciones virtuales utilizadas por los procesos hacia las ubicaciones correspondientes en la memoria física. Microsoft señala que la memoria virtual está en uso incluso cuando la RAM instalada es suficiente para los procesos que se están ejecutando.

Esto permite además que cada proceso cuente con su propio espacio de direcciones virtuales aislado: un proceso no puede acceder ni modificar directamente la memoria perteneciente a otro proceso (Microsoft Learn, s.f.-a).

Windows organiza la memoria virtual y física en unidades llamadas páginas. En arquitecturas x86, estas páginas son tradicionalmente de 4 KB. Cuando el sistema necesita liberar memoria física, puede desplazar determinadas páginas de RAM y respaldar su contenido en almacenamiento, según el tipo de página. El archivo utilizado para este propósito se denomina `pagefile.sys`, y se ubica en disco (Microsoft Learn, s.f.-a).

Una forma sencilla de resumir esta relación es: **RAM** → páginas utilizadas activamente; **disco** / **pagefile.sys** → respaldo de contenido paginable que no necesita permanecer en RAM.

Es importante evitar la simplificación de que "el pagefile es RAM adicional": técnicamente forma parte de los mecanismos de administración de memoria y ayuda a que páginas modificadas poco utilizadas puedan salir de la memoria física. La memoria comprometida disponible en el sistema está relacionada tanto con la RAM como con el archivo de paginación.

3.1.2 Herramientas de monitoreo utilizadas

Para observar el comportamiento de la memoria se utilizaron tres herramientas nativas de Windows Server: Administrador de Tareas, Monitor de Recursos y Monitor de Rendimiento (Performance Monitor). Microsoft recomienda contadores como Memory\Available MBytes, Memory\% Committed Bytes In Use y Process(*)\Working Set para analizar la utilización de memoria (Microsoft Learn, s.f.-b).

3.1.3 Metodología del experimento

Se diseñó un experimento en tres etapas: estado inicial, aumento controlado del consumo de memoria mediante un programa desarrollado en C# (.NET), y observación de los cambios producidos. La evidencia se capturó con el Administrador de Tareas, el Monitor de Recursos y el Monitor de Rendimiento.

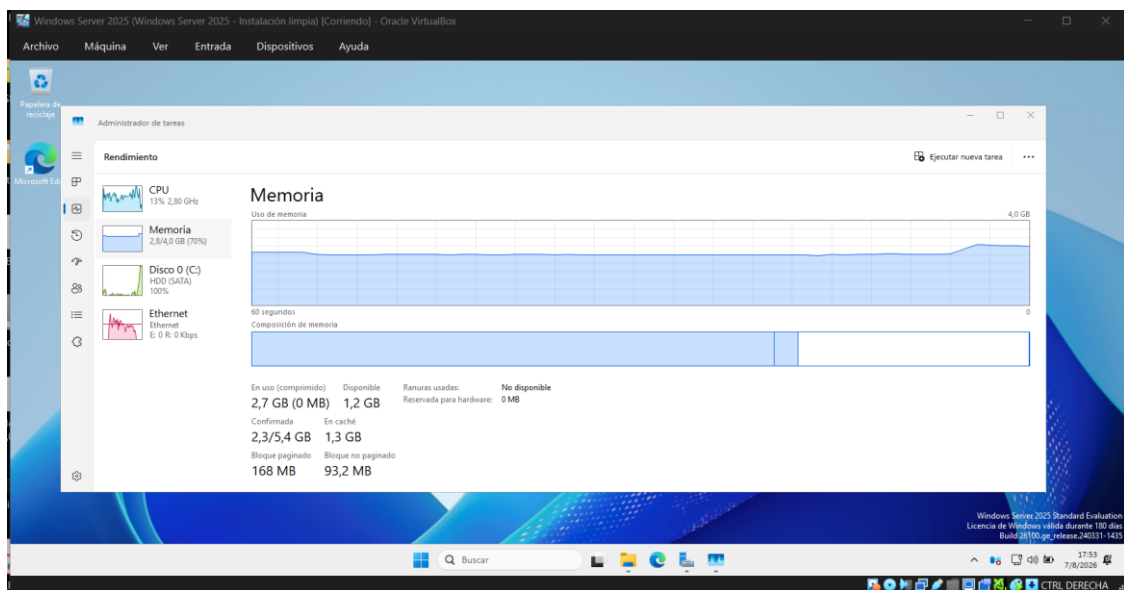


Figura 16. Estado inicial de memoria observado en el Administrador de Tareas.

En esta primera vista, "Confirmada 2,3/5,4 GB" no significa que la VM tenga 5,4 GB de RAM: los 4 GB visibles corresponden a la RAM física asignada a la máquina virtual, mientras que el segundo valor corresponde al límite de memoria comprometida (commit limit), que depende principalmente de la RAM y del archivo de paginación; por eso puede superar los 4 GB físicos. De igual forma, "Bloque paginado" no equivale a "168 MB están en este momento dentro de

pagefile.sys": se trata del paged pool del kernel, es decir, memoria del sistema que puede ser paginada cuando corresponda.

Para poder ejecutar el programa de prueba desarrollado en C#, se instaló el SDK de .NET dentro de la máquina virtual y se comprobó su instalación desde PowerShell.

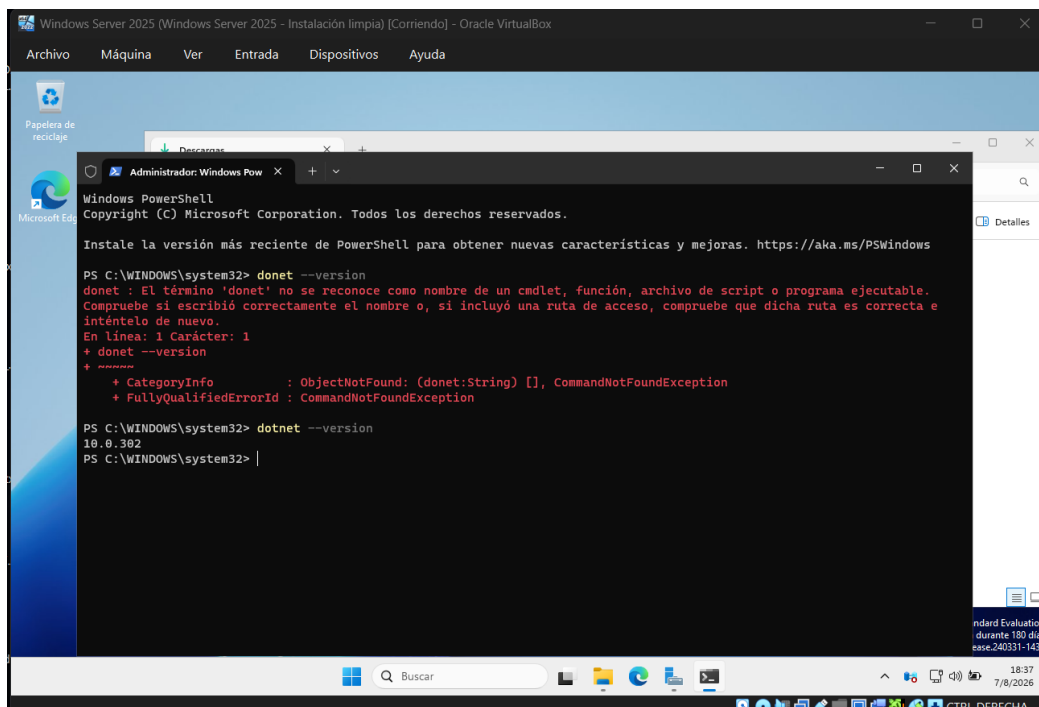


Figura 17. Verificación de la instalación del SDK de .NET en PowerShell.

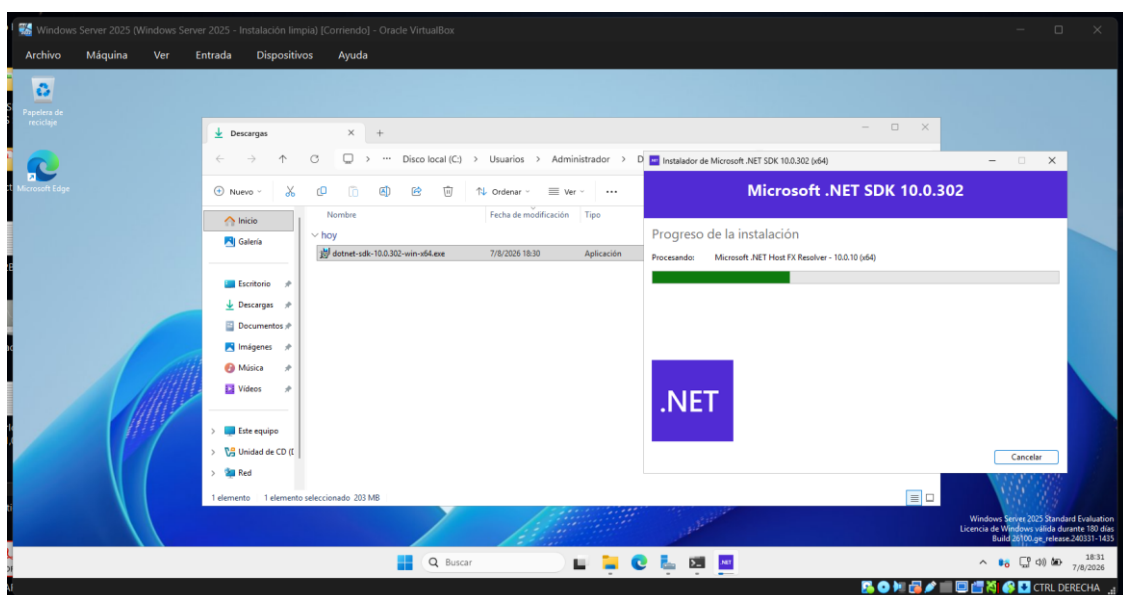
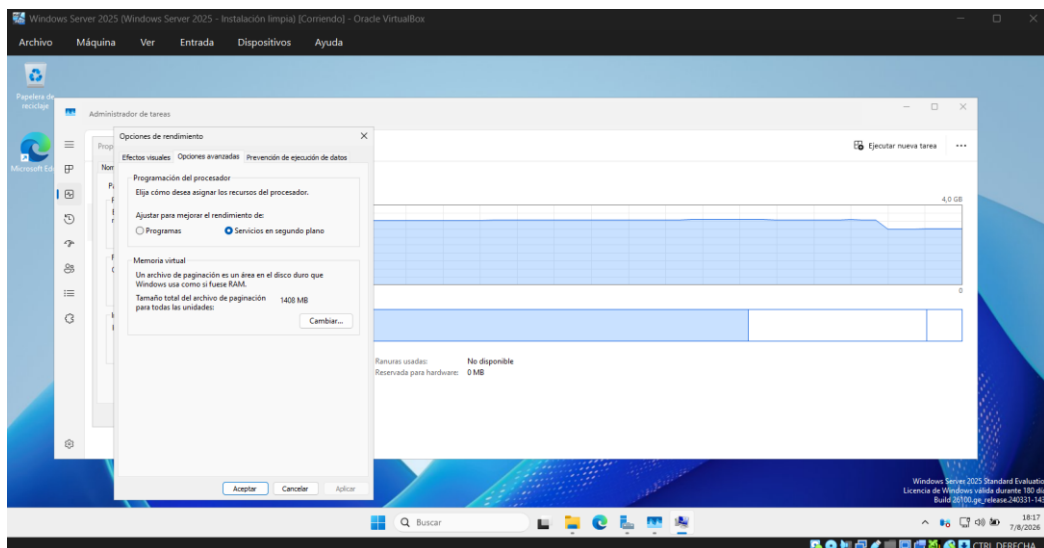
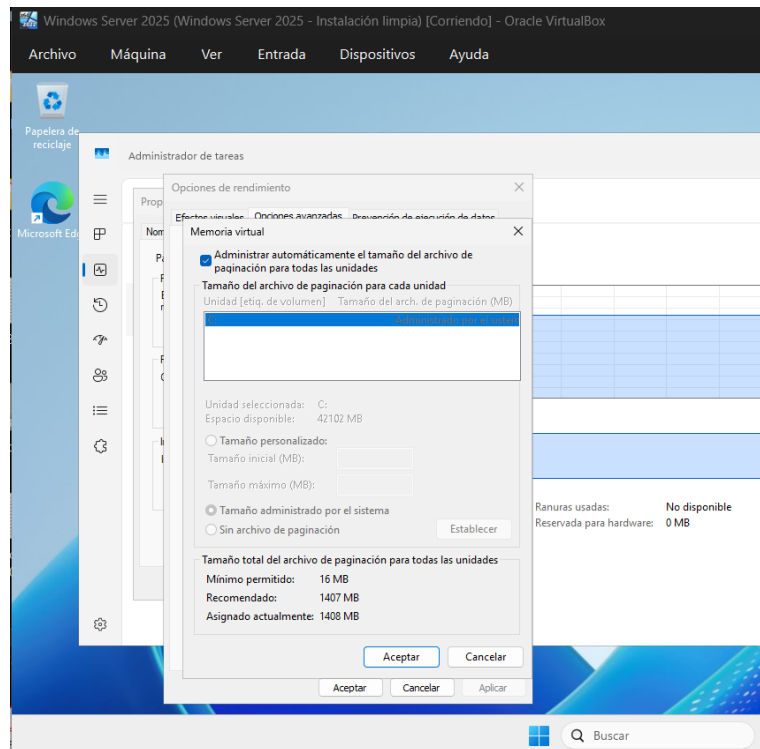


Figura 17.5 Verificación de la instalación del SDK de .NET en PowerShell.

3.1.4 Evidencia 1 — Configuración del archivo de paginación

Windows Server administra un archivo de paginación como parte de su gestión de memoria virtual. En la máquina virtual se comprobó que Windows administra pagefile.sys de forma automática; en este caso, el sistema asignó 1408 MB en la unidad C:, y su tamaño puede ser gestionado dinámicamente por el propio sistema operativo. Esta relación —los 4 GB de RAM asignados a la VM más los 1408 MB de pagefile— permite entender el límite de memoria comprometida cercano a 5,4 GB observado anteriormente.





Figuras 18 y 19. Configuración de memoria virtual y archivo de paginación en Windows Server 2025.

Figura 20. Configuración final del archivo de paginación aplicada.

3.1.5 Evidencia 2 — Prueba de reserva de memoria de 500 MB

Se desarrolló una aplicación en C# que reserva 500 MB de memoria y la mantiene asignada mientras se observa el comportamiento del sistema. Durante la ejecución se evidenció un incremento de la memoria comprometida, así como cambios en la disponibilidad y utilización de memoria mostrados por el Administrador de Tareas.

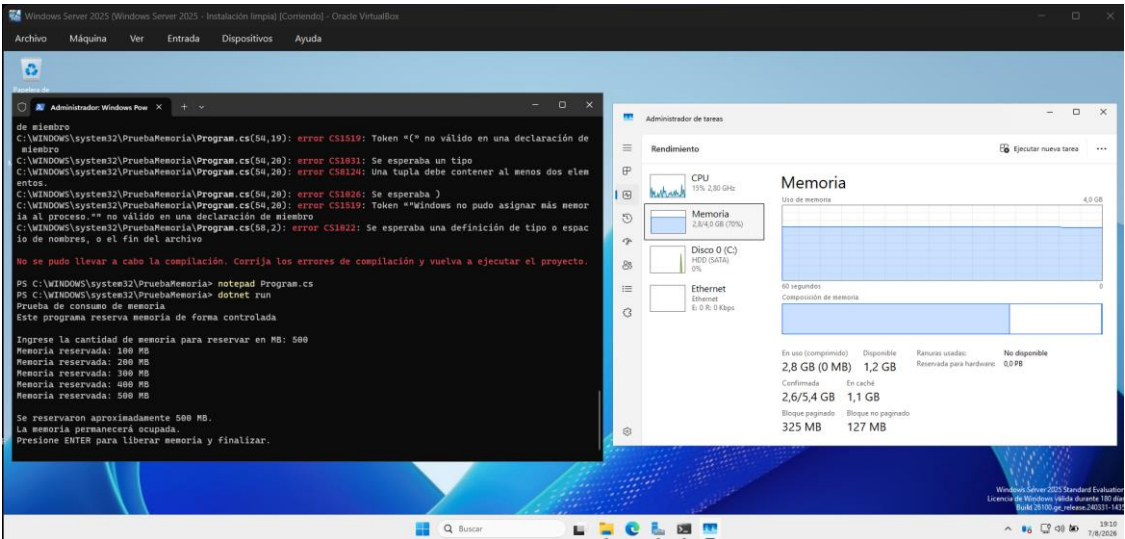


Figura 21. Administrador de Tareas durante la reserva de 500 MB.

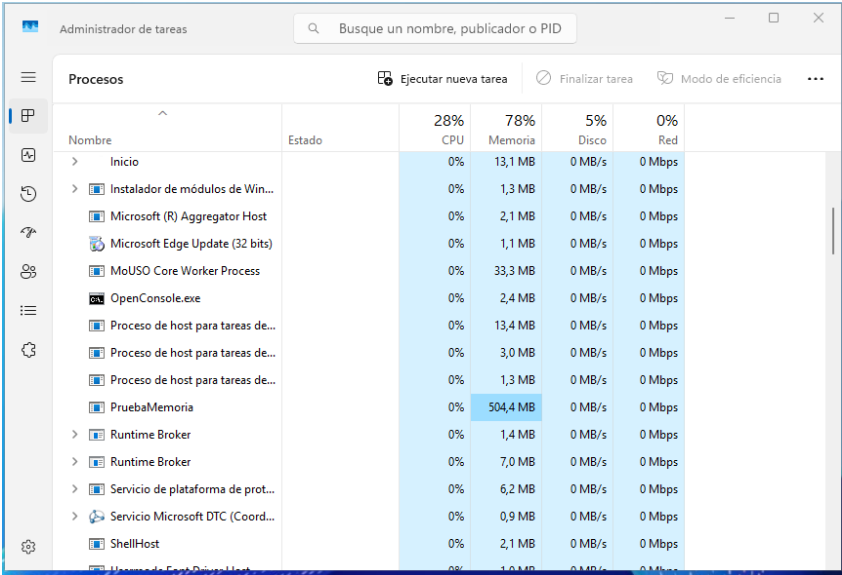
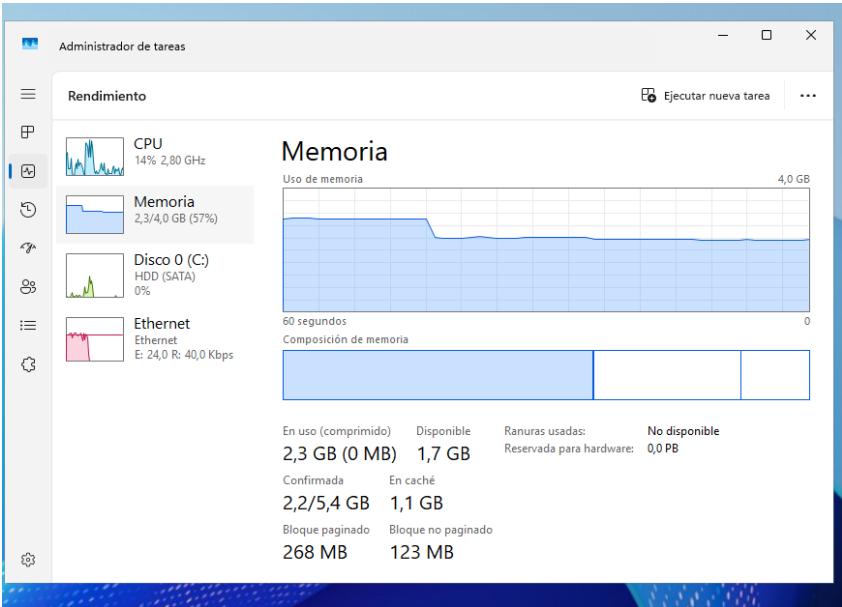
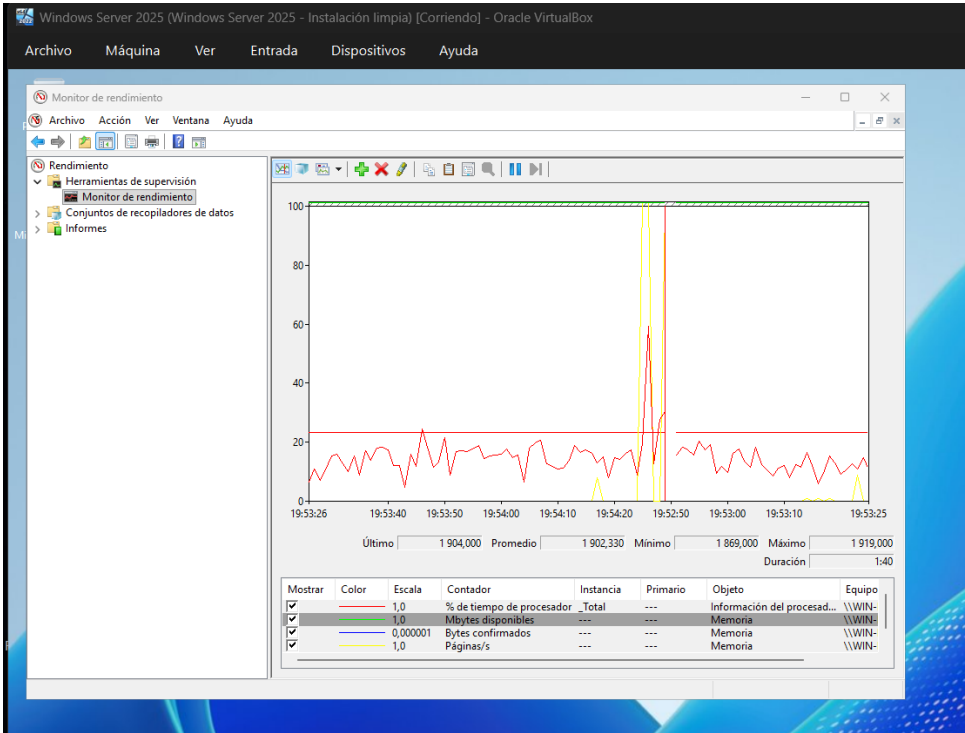
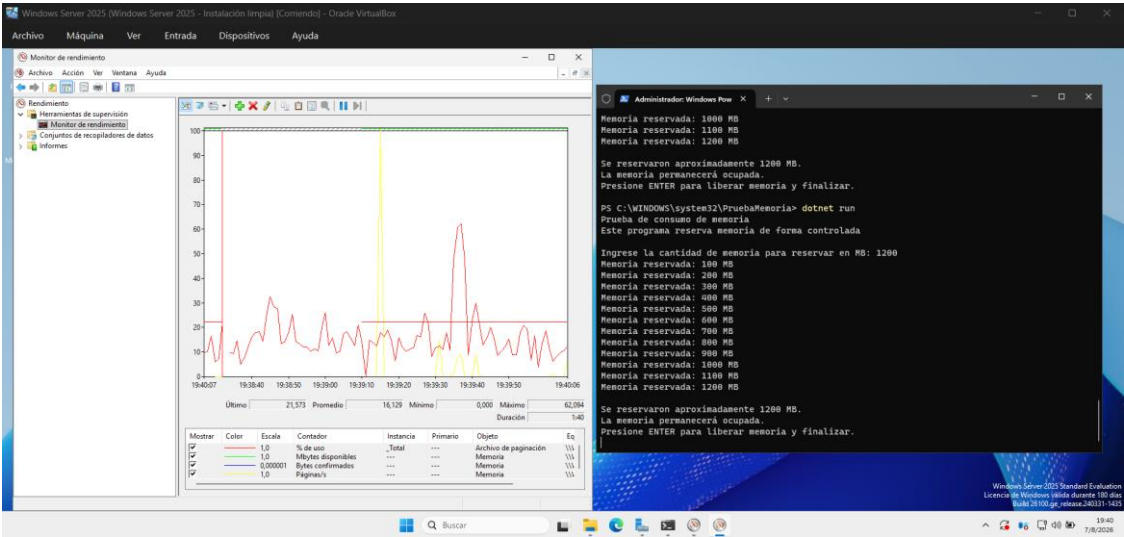


Figura 22. Programa en C# utilizado para reservar memoria de forma controlada.



3.1.6 Evidencia 3 — Prueba de reserva de memoria de 1200 MB con Monitor de Rendimiento

Se desarrolló un segundo programa en C# que reserva memoria RAM de forma controlada, permitiendo indicar la cantidad deseada por consola. Al asignarle 1200 MB, se utilizó el Monitor de Rendimiento de Windows Server para observar cómo disminuye la memoria disponible y aumenta la actividad de paginación mientras la memoria permanece reservada. Al finalizar el programa, Windows libera los recursos y la memoria disponible vuelve a aumentar.



Figuras 26 y 27. Configuración del Monitor de Rendimiento y preparación de la prueba en consola.

Prueba — antes

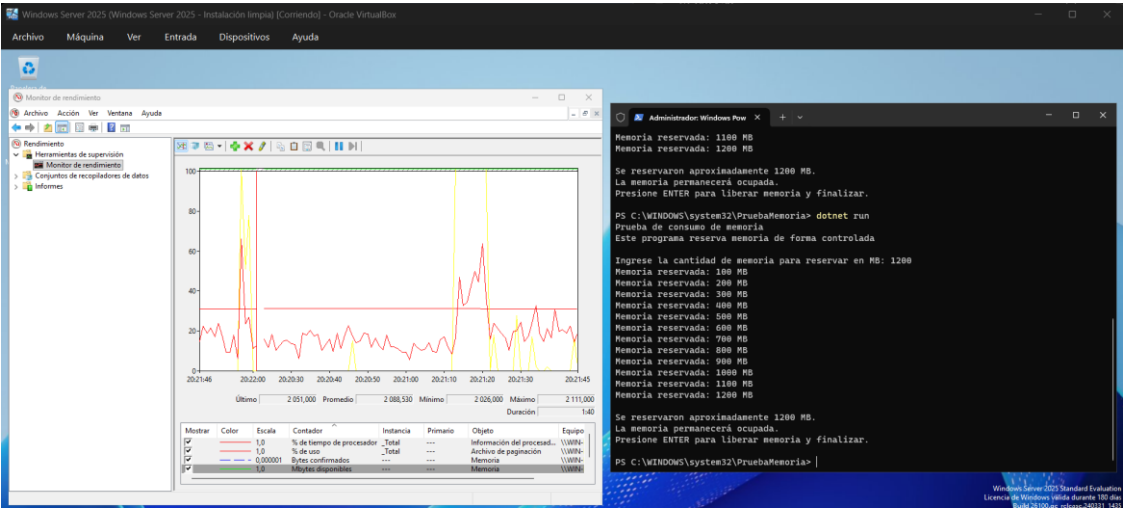
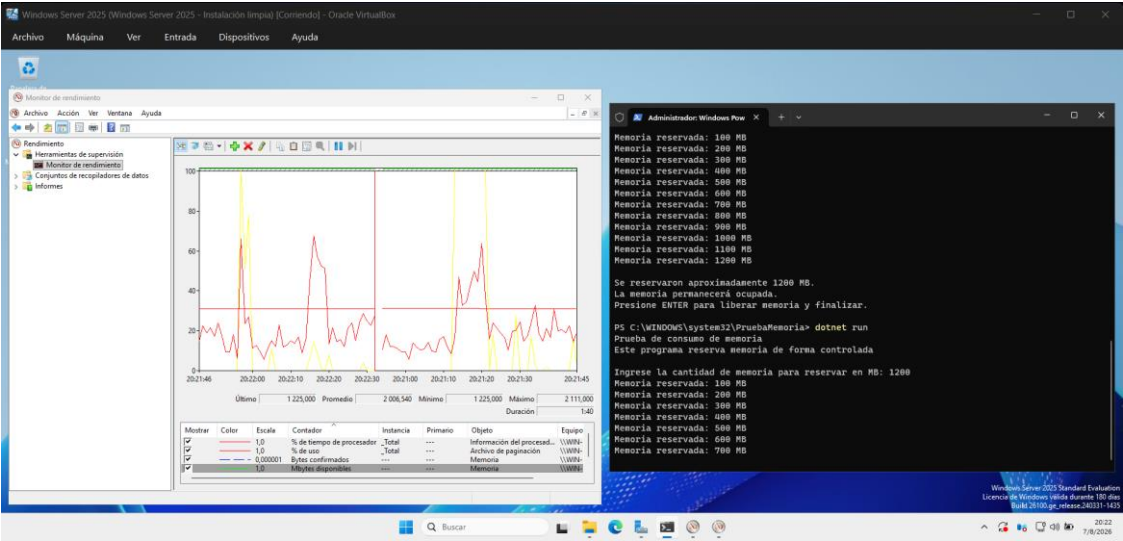
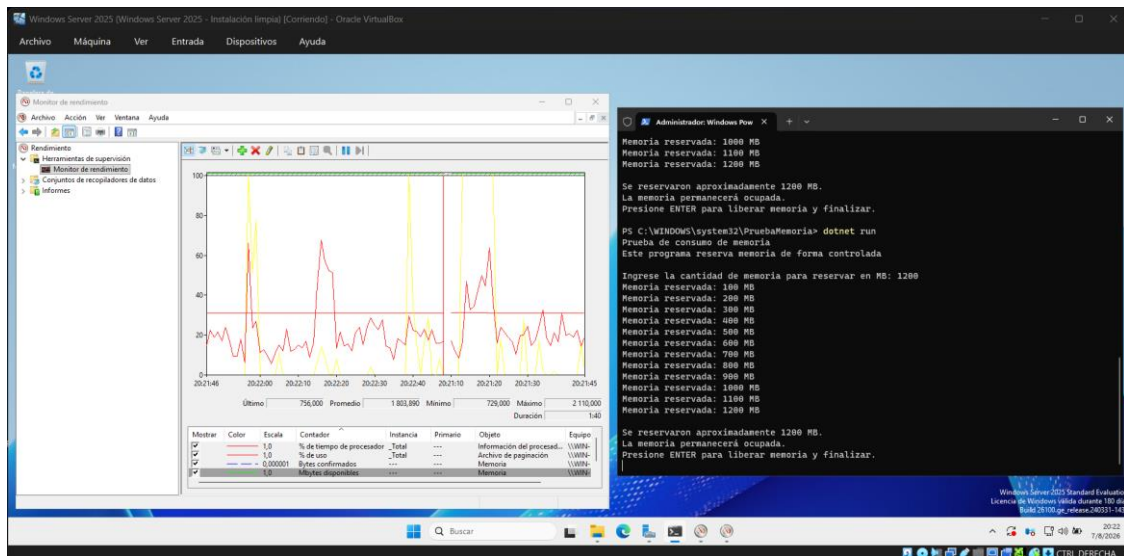


Figura 28. Estado de la memoria antes de iniciar la reserva de 1200 MB.

Prueba — durante





Figuras 29 y 30. Consumo de memoria y actividad de paginación mientras el programa mantiene reservados los 1200 MB.

Prueba — después

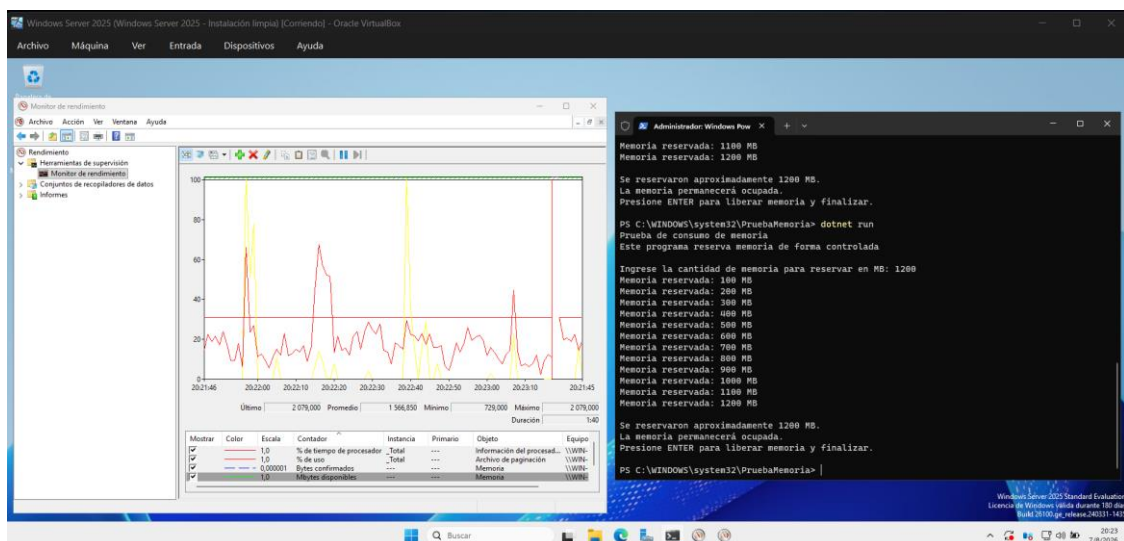


Figura 31. Memoria disponible recuperándose tras finalizar el programa.

Para verificar la consistencia del comportamiento observado, la prueba se repitió una segunda vez dentro de la misma sesión, obteniendo resultados equivalentes: el consumo de memoria vuelve a subir hasta los 1200 MB reservados y se libera nuevamente al finalizar la ejecución, confirmando que el comportamiento no fue un caso aislado.

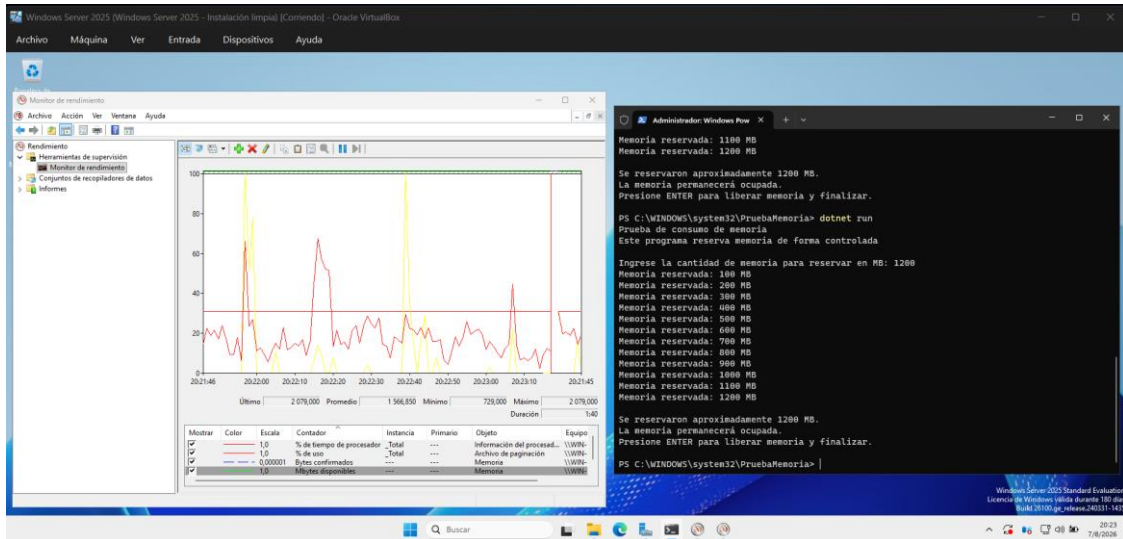


Figura 32. Segunda ejecución de la prueba de reserva de 1200 MB, confirmando la consistencia de los resultados.

3.1.7 Preguntas de la rúbrica — Windows Server

¿Cómo maneja este SO la paginación? Mediante un archivo de paginación (pagefile.sys) administrado dinámicamente, en páginas de 4 KB, combinando memoria física y memoria comprometida (commit limit).

¿Qué herramientas de usuario permiten visualizarlo? Administrador de Tareas, Monitor de Recursos y Monitor de Rendimiento (Performance Monitor), con contadores como Memory\Available MBytes y Memory\% Committed Bytes In Use.

¿Cuál es la API o sistema de llamadas para sockets y pipes en Windows Server?

Para sockets, Windows Server usa la API de Winsock, que ofrece funciones como socket(), bind(), listen(), accept(), connect(), send() y recv(). En C# esto se maneja con las clases TcpListener y TcpClient de System.Net.Sockets, que por dentro usan Winsock. Para las tuberías con nombre (Named Pipes), Windows usa funciones como CreateNamedPipe(), ConnectNamedPipe(), WaitNamedPipe(), ReadFile() y WriteFile(). En C# esto se maneja con las clases NamedPipeServerStream y NamedPipeClientStream de System.IO.Pipes.

⚠ PENDIENTE — Sección a completar por el resto del grupo

- *¿Cómo se implementan procesos e hilos en este entorno? (parte de fabricio, sección 3.5)*
- *¿Qué diferencias existen entre semáforo y mutex en este SO? (parte de fabricio, sección 3.6)*

3.2 Sockets (comunicación entre procesos en red)**Marco teórico**

Los sockets en Windows funcionan sobre Winsock, que es la interfaz que usa el sistema operativo para manejar comunicación por red. Permite que dos programas se conecten entre sí usando una dirección IP y un puerto, y una vez conectados pueden mandarse información en ambas direcciones.

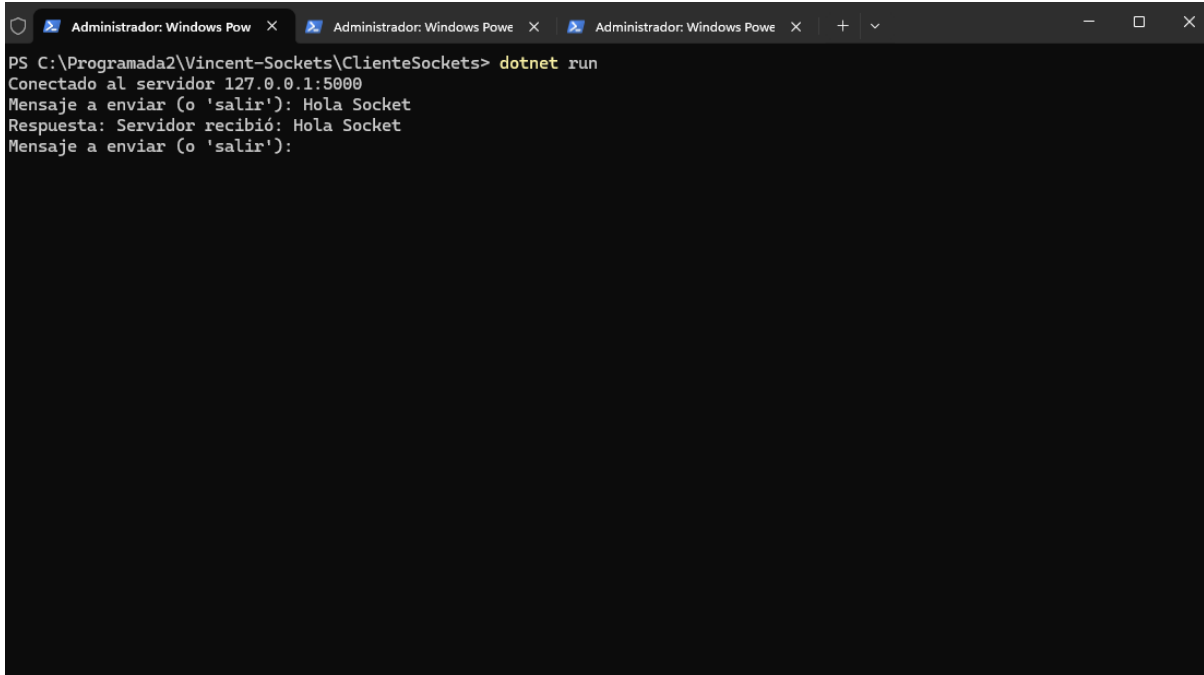
En C# no se necesita usar Winsock directamente porque .NET ya trae las clases `TcpListener` (para el servidor) y `TcpClient` (para el cliente), que hacen todo el trabajo de bajo nivel.

Un socket TCP funciona con conexión: primero el servidor se queda escuchando en un puerto, y el cliente se conecta a esa IP y puerto. Después de que se conectan, pueden mandarse mensajes hasta que uno de los dos cierre la conexión.

Metodología del experimento

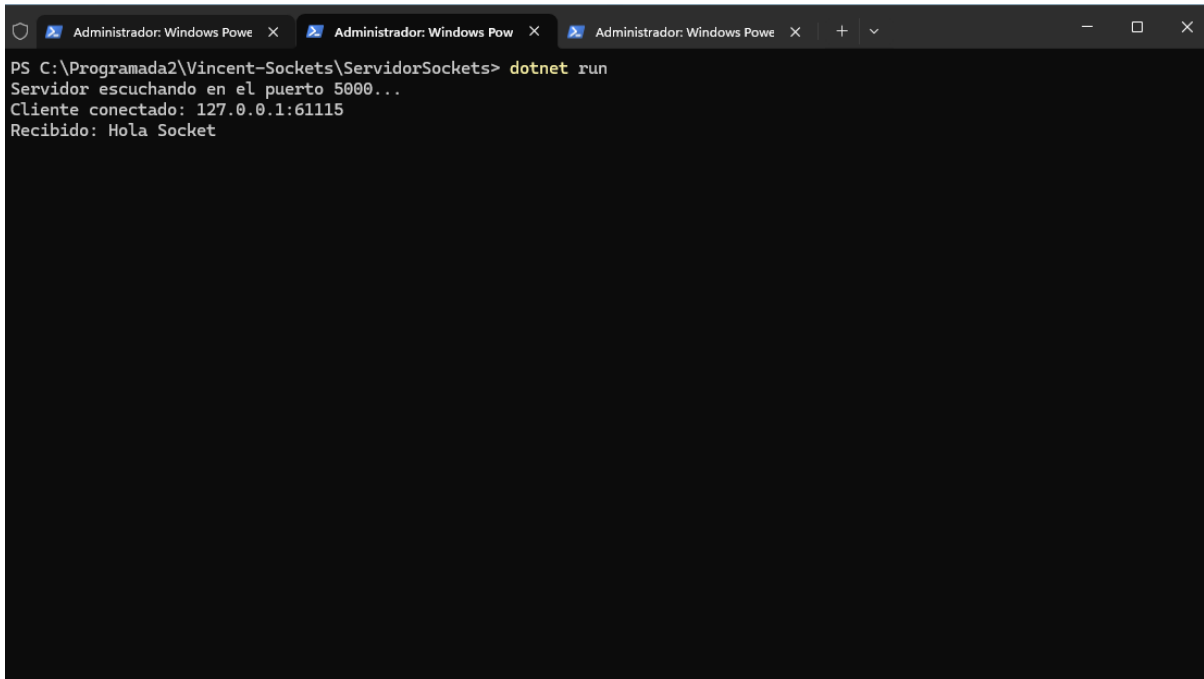
Se hicieron dos programas en C#: un servidor (`ServidorSockets`) que escucha en el puerto 5000, y un cliente (`ClienteSockets`) que se conecta a él. Los dos se corrieron dentro de la misma VM de Windows Server, usando la IP local (127.0.0.1). Se mandó un mensaje desde el cliente y se comprobó que el servidor lo recibiera y respondiera, y también se revisó la conexión con `netstat`.

Evidencia 1 — Cliente



```
PS C:\Programada2\Vincent-Sockets\ClienteSockets> dotnet run
Conectado al servidor 127.0.0.1:5000
Mensaje a enviar (o 'salir'): Hola Socket
Respuesta: Servidor recibió: Hola Socket
Mensaje a enviar (o 'salir'):
```

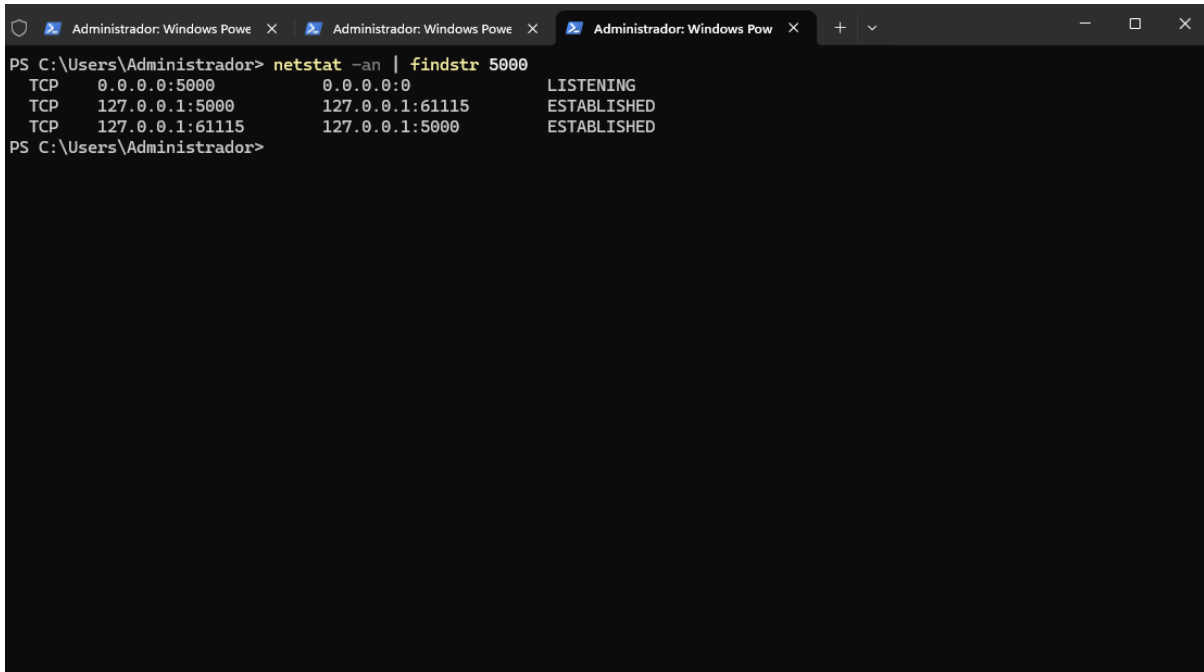
Evidencia 2 — Servidor



```
PS C:\Programada2\Vincent-Sockets\ServidorSockets> dotnet run
Servidor escuchando en el puerto 5000...
Cliente conectado: 127.0.0.1:61115
Recibido: Hola Socket
```

Evidencia 3 — netstat

Se usó el comando `netstat -an | findstr 5000` para ver la conexión. Se ve una línea en estado LISTENING (el servidor esperando conexiones) y dos líneas en ESTABLISHED, que son los dos lados de la conexión ya hecha entre el cliente y el servidor.



```

PS C:\Users\Administrador> netstat -an | findstr 5000
TCP    0.0.0.0:5000      0.0.0.0:0        LISTENING
TCP    127.0.0.1:5000    127.0.0.1:61115  ESTABLISHED
TCP    127.0.0.1:61115  127.0.0.1:5000   ESTABLISHED
PS C:\Users\Administrador>

```

3.3 Tuberías (pipes y pipes con nombre)

Marco teórico

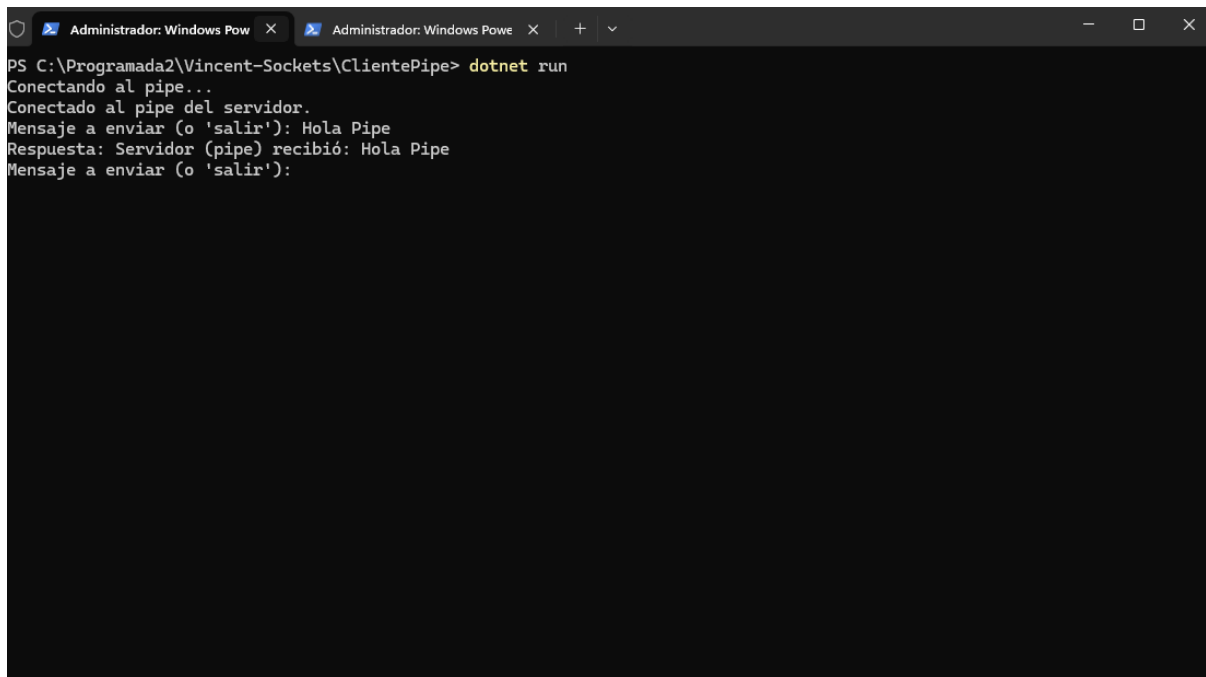
Los Named Pipes son una forma que tiene Windows para que dos procesos se comuniquen entre sí sin necesitar una IP ni un puerto como los sockets. Se identifican con un nombre (algo como `\\pipe\nombre`) y el sistema operativo se encarga de manejar el canal.

En C# se usan las clases `NamedPipeServerStream` (para el que crea el pipe y espera) y `NamedPipeClientStream` (para el que se conecta). El servidor llama a `WaitForConnection()` para esperar a que alguien se conecte, y el cliente llama a `Connect()` usando el nombre del pipe. Una vez conectados funcionan parecido a los sockets: se puede leer y escribir información en los dos sentidos.

Metodología del experimento

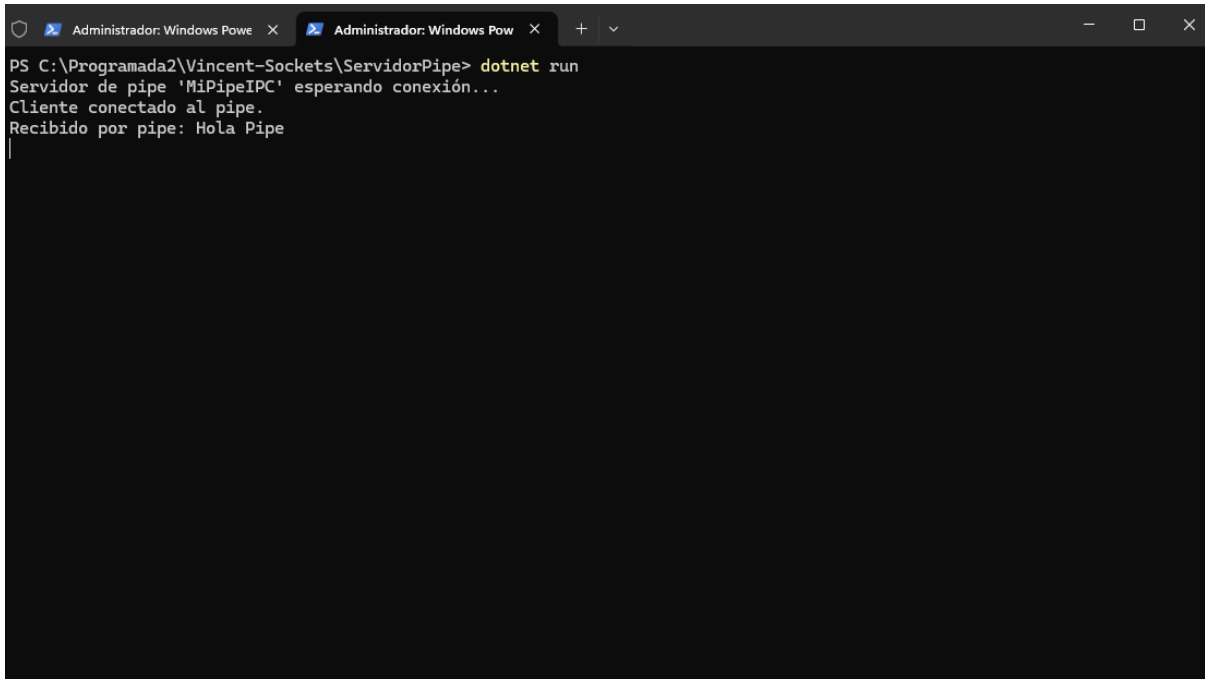
Se hizo un servidor de pipe (ServidorPipe) que crea un pipe llamado MiPipeIPC y espera a que alguien se conecte, y un cliente (ClientePipe) que se conecta a ese pipe. Se mandó un mensaje desde el cliente y se comprobó que el servidor lo recibiera bien.

Evidencia 1 — Cliente



```
PS C:\Programada2\Vincent-Sockets\ClientePipe> dotnet run
Conectando al pipe...
Conectado al pipe del servidor.
Mensaje a enviar (o 'salir'): Hola Pipe
Respuesta: Servidor (pipe) recibió: Hola Pipe
Mensaje a enviar (o 'salir'):
```

Evidencia 2 — Servidor



```

PS C:\Programada2\Vincent-Sockets\ServidorPipe> dotnet run
Servidor de pipe 'MiPipeIPC' esperando conexión...
Cliente conectado al pipe.
Recibido por pipe: Hola Pipe

```

Nota: los Named Pipes no aparecen en netstat porque no usan el protocolo TCP/IP como los sockets, por lo que no se puede verificar con ese comando.

3.4 Mensajes entre procesos (IPC basado en colas o memoria compartida)

Marco teórico

La memoria compartida es un mecanismo de IPC donde dos procesos distintos pueden acceder a la misma parte de memoria RAM. Es más rápido que otros métodos porque los procesos leen y escriben directamente ahí, sin tener que pasar la información por el sistema operativo cada vez.

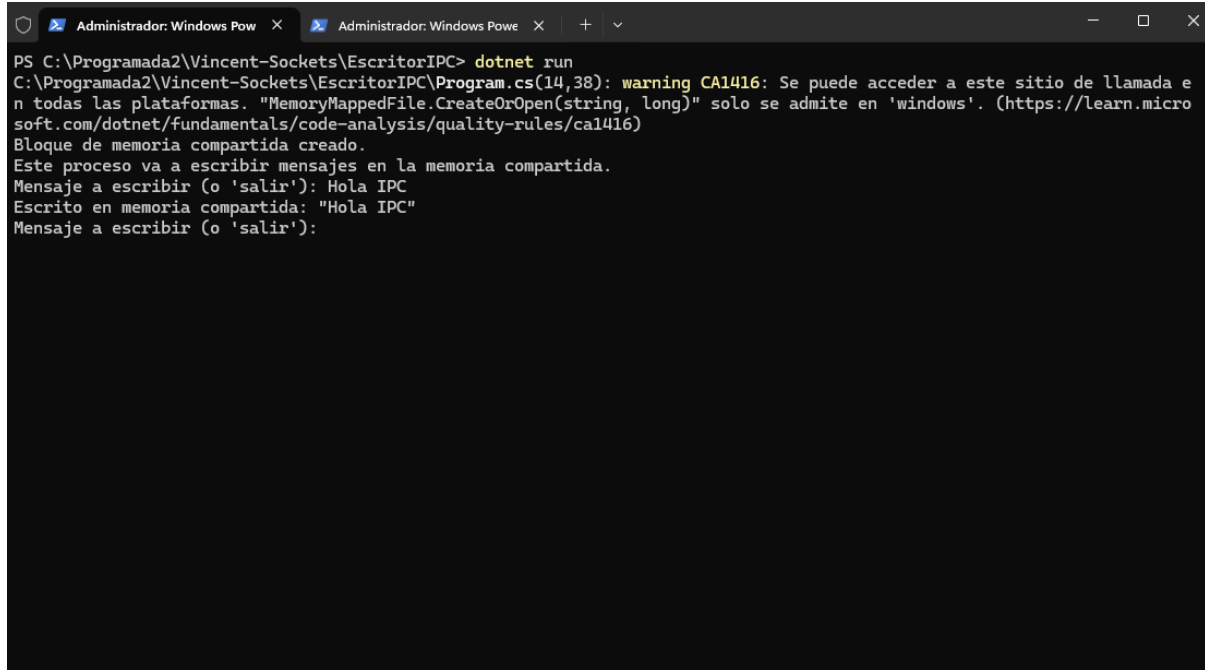
En Windows esto se hace con los Memory-Mapped Files, y en C# se maneja con la clase `MemoryMappedFile` de `System.IO.MemoryMappedFiles`. Un proceso crea el bloque de memoria con `CreateOrOpen()`, y otro proceso se conecta a ese mismo bloque usando `OpenExisting()` y el mismo nombre.

Como puede pasar que dos procesos intenten escribir al mismo tiempo y se genere un error (condición de carrera), se usó un Mutex para asegurar que solo un proceso pueda usar la memoria compartida a la vez mientras escribe o lee.

Metodología del experimento

Se hizo un programa escritor (`EscritorIPC`) que crea el bloque de memoria compartida `MiMemoriaCompartidaIPC` y escribe un mensaje ahí protegido con un Mutex, y un programa lector (`LectorIPC`) que se conecta a ese mismo bloque y lee lo que el escritor dejó guardado.

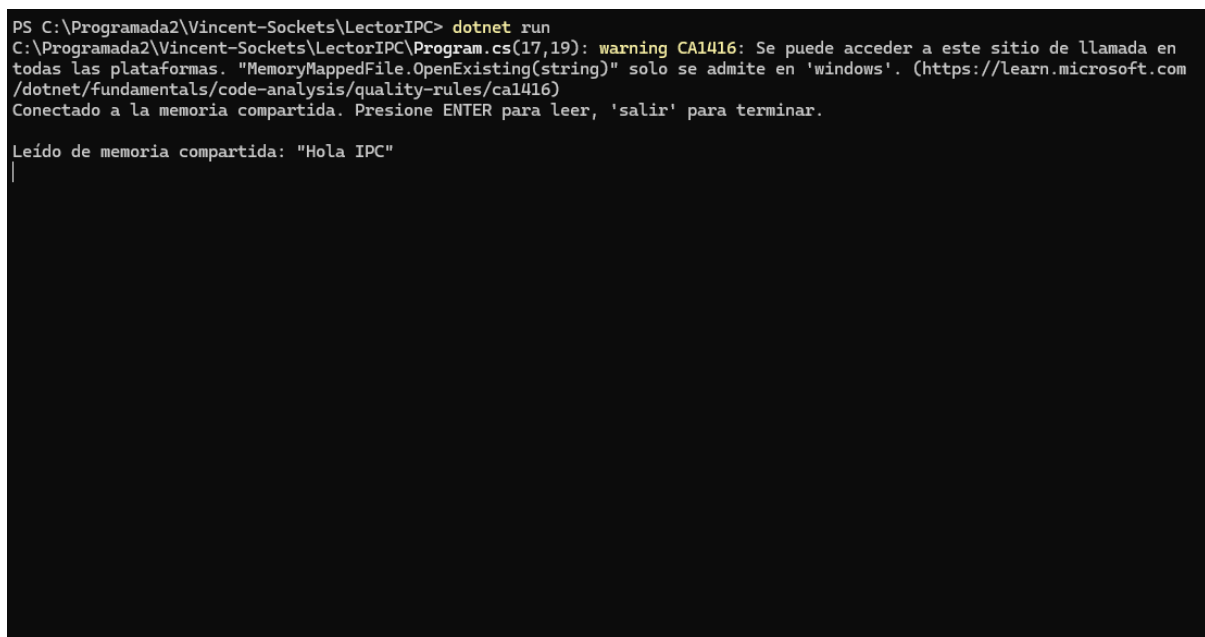
Evidencia 1 — Escritor



```

PS C:\Programada2\Vincent-Sockets\EscritorIPC> dotnet run
C:\Programada2\Vincent-Sockets\EscritorIPC\Program.cs(14,38): warning CA1416: Se puede acceder a este sitio de llamada en todas las plataformas. "MemoryMappedFile.CreateOrOpen(string, long)" solo se admite en 'windows'. (https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/ca1416)
Bloque de memoria compartida creado.
Este proceso va a escribir mensajes en la memoria compartida.
Mensaje a escribir (o 'salir'): Hola IPC
Escrito en memoria compartida: "Hola IPC"
Mensaje a escribir (o 'salir'):
```

Evidencia 2 — Lector



```

PS C:\Programada2\Vincent-Sockets\LectorIPC> dotnet run
C:\Programada2\Vincent-Sockets\LectorIPC\Program.cs(17,19): warning CA1416: Se puede acceder a este sitio de llamada en todas las plataformas. "MemoryMappedFile.OpenExisting(string)" solo se admite en 'windows'. (https://learn.microsoft.com/dotnet/fundamentals/code-analysis/quality-rules/ca1416)
Conectado a la memoria compartida. Presione ENTER para leer, 'salir' para terminar.

Leído de memoria compartida: "Hola IPC"
|
```

3.5 Creación de procesos e hilos (Fork o su equivalente)

Responsable: fabriico.

⚠ PENDIENTE — Sección a completar por el resto del grupo

- Documentar cómo se implementan procesos e hilos en Windows Server (equivalente funcional a `fork()`, p. ej. `CreateProcess/CreateThread` o `Process.Start/Thread` en .NET).
- Desarrollar un programa que cree procesos o hilos de forma controlada.
- Capturar evidencia con *Process Explorer* o *PowerShell*.

3.6 Semáforos y Mutex

Responsable: ffabricio.

⚠ PENDIENTE — Sección a completar por el resto del grupo

- Explicar las diferencias entre semáforo y mutex en Windows Server.
- Desarrollar un programa que use *Semaphore* y *Mutex* para sincronización entre hilos.
- Incluir evidencia de ejecución y el código comentado.

4. Comparación con Otro Sistema Operativo

A modo de comparación inicial (a validar y ampliar por el grupo, en particular por quien documente el sistema Linux/Kali), se contrastan tres aspectos entre Windows Server y Linux:

Aspecto	Windows Server	Linux (Kali)
Paginación / memoria virtual	Usa <code>pagefile.sys</code> , tamaño gestionado automáticamente por el sistema; páginas de 4 KB; visible desde Task Manager, Resource Monitor y Performance Monitor.	Usa una partición o archivo swap; páginas también de 4 KB típicamente; visible con <code>free</code> , <code>vmstat</code> , <code>top</code> o <code>/proc/meminfo</code> .
Comunicación entre procesos	Named Pipes, sockets Winsock, MSMQ; se inspecciona con TCPView y netstat.	Pipes y named pipes (FIFO), sockets Berkeley (<code>socket()</code> , <code>pipe()</code>); se inspecciona con <code>ipcs</code> , <code>netstat/ss</code> .
Procesos e hilos	Se crean con <code>CreateProcess/CreateThread</code> (o su equivalente en .NET); administrados por el Process/Thread Scheduler de	Se crean con <code>fork()</code> y <code>pthread_create()</code> ; administrados por el scheduler de Linux (CFS); se observan con <code>ps</code> , <code>htop</code> o <code>strace</code> .

	Windows; se observan con Process Explorer.	
--	--	--

⚠ PENDIENTE — Sección a completar por el resto del grupo

- *Confirmar y ampliar la comparación con evidencia práctica del otro sistema operativo (mínimo 1 SO adicional, según la rúbrica).*
- *Agregar capturas de pantalla del otro sistema operativo que respalden la comparación.*

5. Código Fuente

El código fuente de los programas desarrollados en C# para las pruebas de memoria virtual (reserva de 500 MB y de 1200 MB) se muestra en las capturas de las figuras 22 y 27 de la sección 3.1. Se recomienda anexar aquí el código completo en formato de texto (no solo como imagen), para facilitar su lectura y evaluación.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Threading;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Console.WriteLine("Prueba de consumo de memoria");
```

```
        Console.WriteLine("Este programa reserva memoria de forma controlada");
```

```
Console.WriteLine();
```

```
Console.Write("Ingrese la cantidad de memoria para reservar en MB: ");
```

```
if (!int.TryParse(Console.ReadLine(), out int megabytes) || megabytes <= 0)
```

```
{
```

```
    Console.WriteLine("Valor incorrecto");
```

```
    return;
```

```
}
```

```
var bloques = new List<byte[]>();
```

```
try
```

```
{
```

```
    for (int i = 0; i < megabytes; i++)
```

```
    {
```

```
        // Reserva 1 MB por iteración
```

```
        byte[] bloque = new byte[1024 * 1024];
```

```
// Escribe en toda la memoria para forzar su uso real

Array.Fill(bloque, (byte)1);


// Mantiene la memoria ocupada

bloques.Add(bloque);


if ((i + 1) % 100 == 0)

{

    Console.WriteLine($"Memoria reservada: {i + 1} MB");

}


Thread.Sleep(10);

}


Console.WriteLine();

Console.WriteLine($"Se reservaron aproximadamente {megabytes} MB.");

Console.WriteLine("La memoria permanecerá ocupada.");

Console.WriteLine("Presione ENTER para liberar memoria y finalizar.");
```

```
        Console.ReadLine();

    }

    catch (OutOfMemoryException)

    {

        Console.WriteLine();

        Console.WriteLine("Windows no pudo asignar más memoria al proceso.");

    }

}

}
```

Servidor de Sockets (ServidorSockets/Program.cs)

```
using System;
```

```
using System.Net;
```

```
using System.Net.Sockets;
```

```
using System.Text;
```

```
class ServidorSockets
```

```
{
```

```
    static void Main()
```

```
{
```

```
int puerto = 5000;

// Se crea el socket servidor y se pone a escuchar en el puerto 5000

TcpListener servidor = new TcpListener(IPAddress.Any, puerto);

servidor.Start();

Console.WriteLine($"Servidor escuchando en el puerto {puerto}...");

while (true)

{

    // El servidor espera a que un cliente se conecte

    using TcpClient cliente = servidor.AcceptTcpClient();

    Console.WriteLine($"Cliente conectado: {cliente.Client.RemoteEndPoint}");

    using NetworkStream stream = cliente.GetStream();

    byte[] buffer = new byte[1024];

    while (cliente.Connected)

    {

        int bytesLeidos;
```

```
try

{

    // Se leen los datos que manda el cliente

    bytesLeidos = stream.Read(buffer, 0, buffer.Length);

}

catch

{

    break;

}

if (bytesLeidos == 0) break;

string mensaje = Encoding.UTF8.GetString(buffer, 0, bytesLeidos);

Console.WriteLine($"Recibido: {mensaje}");

// Se responde al cliente confirmando el mensaje recibido

string respuesta = $"Servidor recibió: {mensaje}";

byte[] datosRespuesta = Encoding.UTF8.GetBytes(respuesta);

stream.Write(datosRespuesta, 0, datosRespuesta.Length);
```

```
        if (mensaje.Trim().ToLower() == "salir") break;

    }

    Console.WriteLine("Cliente desconectado.");

}

}

}
```

Cliente de Sockets (ClienteSockets/Program.cs)

```
using System;

using System.Net.Sockets;

using System.Text;

class ClienteSockets

{

    static void Main()

    {

        string ip = "127.0.0.1";

        int puerto = 5000;
```



```
// Se crea el socket cliente y se conecta al servidor

using TcpClient cliente = new TcpClient();

cliente.Connect(ip, puerto);

Console.WriteLine($"Conectado al servidor {ip}:{puerto}");


using NetworkStream stream = cliente.GetStream();

byte[] buffer = new byte[1024];


while (true)

{

    Console.Write("Mensaje a enviar (o 'salir'): ");

    string? mensaje = Console.ReadLine();

    if (string.IsNullOrEmpty(mensaje)) continue;


    // Se envía el mensaje al servidor

    byte[] datos = Encoding.UTF8.GetBytes(mensaje);

    stream.Write(datos, 0, datos.Length);
```

```
if (mensaje.Trim().ToLower() == "salir") break;
```

```
// Se espera y se lee la respuesta del servidor
```

```
int bytesLeidos = stream.Read(buffer, 0, buffer.Length);
```

```
string respuesta = Encoding.UTF8.GetString(buffer, 0, bytesLeidos);
```

```
Console.WriteLine($"Respuesta: {respuesta}");
```

```
}
```

```
Console.WriteLine("Desconectado.");
```

```
}
```

```
}
```

Servidor de Pipe (ServidorPipe/Program.cs)

```
using System;
```

```
using System.IO.Pipes;
```

```
using System.Text;
```

```
class ServidorPipe
```

```
{
```

```
    static void Main()
```

```
{

string nombrePipe = "MiPipeIPC";


while (true)

{

    // Se crea el pipe con nombre y se espera a que un cliente se conecte

    using NamedPipeServerStream servidor = new NamedPipeServerStream(

        nombrePipe, PipeDirection.InOut);


    Console.WriteLine($"Servidor de pipe '{nombrePipe}' esperando conexión...");

    servidor.WaitForConnection();

    Console.WriteLine("Cliente conectado al pipe.");


    byte[] buffer = new byte[1024];


    while (servidor.IsConnected)

    {

        int bytesLeidos;

        try
```

```
{  
  
    // Se leen los datos enviados por el cliente a través del pipe  
  
    bytesLeidos = servidor.Read(buffer, 0, buffer.Length);  
  
}  
  
catch  
  
{  
  
    break;  
  
}  
  
if (bytesLeidos == 0) break;  
  
string mensaje = Encoding.UTF8.GetString(buffer, 0, bytesLeidos);  
  
Console.WriteLine($"Recibido por pipe: {mensaje}");  
  
// Se responde al cliente por el mismo canal  
  
string respuesta = $"Servidor (pipe) recibió: {mensaje}";  
  
byte[] datosRespuesta = Encoding.UTF8.GetBytes(respuesta);  
  
servidor.Write(datosRespuesta, 0, datosRespuesta.Length);  
  
servidor.Flush();
```

```
        if (mensaje.Trim().ToLower() == "salir") break;

    }

    Console.WriteLine("Cliente del pipe desconectado.\n");

}

}

}
```

Cliente de Pipe (ClientePipe/Program.cs)

```
using System;

using System.IO.Pipes;

using System.Text;

class ClientePipe

{

    static void Main()

    {

        string nombrePipe = "MiPipeIPC";
```

```
// Se conecta al pipe con nombre creado por el servidor

using NamedPipeClientStream cliente = new NamedPipeClientStream(

    ".", nombrePipe, PipeDirection.InOut);

Console.WriteLine("Conectando al pipe...");

cliente.Connect();

Console.WriteLine("Conectado al pipe del servidor.");


byte[] buffer = new byte[1024];


while (true)

{

    Console.Write("Mensaje a enviar (o 'salir'): ");

    string? mensaje = Console.ReadLine();

    if (string.IsNullOrEmpty(mensaje)) continue;


    // Se envía el mensaje por el pipe

    byte[] datos = Encoding.UTF8.GetBytes(mensaje);

    cliente.Write(datos, 0, datos.Length);
```

```
cliente.Flush();
```

```
if (mensaje.Trim().ToLower() == "salir") break;
```

```
// Se lee la respuesta del servidor
```

```
int bytesLeidos = cliente.Read(buffer, 0, buffer.Length);
```

```
string respuesta = Encoding.UTF8.GetString(buffer, 0, bytesLeidos);
```

```
Console.WriteLine($"Respuesta: {respuesta}");
```

```
}
```

```
Console.WriteLine("Desconectado.");
```

```
}
```

```
}
```

Escritor de memoria compartida (EscritorIPC/Program.cs)

```
using System;
```

```
using System.IO.MemoryMappedFiles;
```

```
using System.Text;
```

```
using System.Threading;
```

```
class EscritorIPC

{

    static void Main()

    {

        string nombreMemoria = "MiMemoriaCompartidaIPC";

        int tamano = 1024;


        // El Mutex evita que dos procesos escriban/lean al mismo tiempo (condición de carrera)

        using Mutex mutex = new Mutex(false, "MiMutexIPC");


        // Se crea (o abre si ya existe) el bloque de memoria compartida

        using MemoryMappedFile mmf = MemoryMappedFile.CreateOrOpen(nombreMemoria,
tamano);


        Console.WriteLine("Bloque de memoria compartida creado.");

        Console.WriteLine("Este proceso va a escribir mensajes en la memoria compartida.");


        while (true)

        {
```



```

Console.Write("Mensaje a escribir (o 'salir'): ");

string? mensaje = Console.ReadLine();

if (string.IsNullOrEmpty(mensaje)) continue;

// Se toma el Mutex antes de escribir para tener acceso exclusivo

mutex.WaitOne();

try

{

    using MemoryMappedViewAccessor accesor = mmf.CreateViewAccessor();

    byte[] datos = Encoding.UTF8.GetBytes(mensaje);

    // Se escribe primero la longitud del mensaje, y luego el mensaje en sí

    accesor.Write(0, datos.Length);

    accesor.WriteArray(4, datos, 0, datos.Length);

    Console.WriteLine($"Escrito en memoria compartida: \"{mensaje}\"");

}

finally

{

```

```
// Se libera el Mutex para que otro proceso pueda usar la memoria

mutex.ReleaseMutex();

}

if (mensaje.Trim().ToLower() == "salir") break;

}

Console.WriteLine("Presione ENTER para liberar la memoria compartida y finalizar.");

Console.ReadLine();

}

}

Lector de memoria compartida (LectorIPC/Program.cs)

using System;

using System.IO.MemoryMappedFiles;

using System.Text;

using System.Threading;

class LectorIPC

{
```

```
static void Main()

{

    string nombreMemoria = "MiMemoriaCompartidaIPC";


    using Mutex mutex = new Mutex(false, "MiMutexIPC");


    MemoryMappedFile mmf;

    try

    {

        // Se conecta al bloque de memoria compartida ya creado por el Escritor

        mmf = MemoryMappedFile.OpenExisting(nombreMemoria);

    }

    catch (Exception)

    {

        Console.WriteLine("No se encontró la memoria compartida. Asegúrese de que el  
Escritor esté corriendo.");

        return;

    }

}
```

```
Console.WriteLine("Conectado a la memoria compartida. Presione ENTER para leer,  
'salir' para terminar.");
```

```
using (mmf)
```

```
{
```

```
while (true)
```

```
{
```

```
string? entrada = Console.ReadLine();
```

```
if (entrada != null && entrada.Trim().ToLower() == "salir") break;
```

```
// Se toma el Mutex antes de leer, para no chocar con una escritura en curso
```

```
mutex.WaitOne();
```

```
try
```

```
{
```

```
using MemoryMappedViewAccessor accesor = mmf.CreateViewAccessor();
```

```
int longitud = accesor.ReadInt32(0);
```

```
if (longitud > 0 && longitud < 1024)
```

```
{
```

```
byte[] datos = new byte[longitud];

accesor.ReadArray(4, datos, 0, longitud);

string mensaje = Encoding.UTF8.GetString(datos);

Console.WriteLine($"Leído de memoria compartida: \"{mensaje}\"");

}

else

{

    Console.WriteLine("(memoria vacía o sin datos válidos)");

}

}

finally

{

    mutex.ReleaseMutex();

}

}

}

Console.WriteLine("Desconectado.");

}
```

```
}
```

⚠ PENDIENTE — Sección a completar por el resto del grupo

- Pegar el código fuente completo (en bloques de texto/monoespaciado) de cada ejemplo: *fork()/equivalente, pipes o sockets, y semáforos/mutex.*
- Comentar cada bloque de código explicando su funcionamiento.

6. Conclusiones

La experimentación práctica realizada sobre Windows Server 2025 permitió confirmar, con evidencia directa, cómo el sistema operativo gestiona la memoria virtual mediante el archivo de paginación (pagefile.sys) y cómo esta gestión se refleja en las métricas de memoria comprometida y disponible observadas con el Administrador de Tareas y el Monitor de Rendimiento.

Se comprobó, además, que el consumo de memoria reportado por las herramientas de monitoreo responde de manera consistente y repetible ante la reserva y liberación controlada de memoria por parte de un proceso, lo cual valida el modelo teórico de paginación descrito por Microsoft.

En cuanto a la comunicación entre procesos, se comprobó de forma práctica que Windows Server ofrece varios mecanismos con propósitos distintos según el escenario. Los sockets permiten comunicación orientada a red usando la IP y el puerto como identificadores, lo cual los hace adecuados cuando los procesos pueden estar en máquinas diferentes; esto se confirmó viendo el estado ESTABLISHED en netstat durante la conexión activa. Los Named Pipes, en cambio, no requieren IP ni puerto, sino que identifican el canal por nombre, siendo una opción

más simple para comunicación local entre procesos en la misma máquina. Por último, la memoria compartida demostró ser el mecanismo más directo de los tres, ya que los procesos acceden al mismo bloque de memoria sin pasar por un protocolo de red ni un canal intermedio, aunque esto exige sincronización explícita (en este caso con un Mutex) para evitar que dos procesos escriban o lean al mismo tiempo y corrompan la información.

⚠ PENDIENTE — Sección a completar por el resto del grupo

- *Agregar conclusiones sobre los componentes de sockets, pipes, IPC, procesos/hilos y semáforos/mutex una vez completados por el resto del grupo.*
- *Agregar una conclusión general sobre la comparación con el otro sistema operativo.*

Referencias

Microsoft. (s.f.-a). Espacios de direcciones virtuales. Microsoft Learn.

<https://learn.microsoft.com/es-es/windows-hardware/drivers/gettingstarted/virtual-address-spaces>

Microsoft. (s.f.-b). Diagnóstico de problemas de rendimiento en Windows Server. Microsoft

Learn. <https://learn.microsoft.com/en-us/troubleshoot/windows-server/performance/troubleshoot-performance-problems-in-windows>

Microsoft. (s.f.-c). VirtualBox — Downloads. Oracle.

<https://www.virtualbox.org/wiki/Downloads>

Microsoft. (s.f.-d). Windows Server 2025 Evaluation Center. [https://www.microsoft.com/es-](https://www.microsoft.com/es-es/evalcenter/download-windows-server-2025)

[es/evalcenter/download-windows-server-2025](https://www.microsoft.com/es-es/evalcenter/download-windows-server-2025)

⚠ PENDIENTE — Sección a completar por el resto del grupo

- *Agregar al menos dos referencias de libros físicos de la biblioteca del CUC (requisito obligatorio de la rúbrica, formato APA 7).*
- *Ejemplo de formato: Apellido, A. A. (Año). Título del libro en cursiva (edición). Editorial.*

Anexos

⚠ PENDIENTE — Sección a completar por el resto del grupo

- *Adjuntar las fotografías del comprobante de préstamo de los libros de la biblioteca del CUC (requisito obligatorio, sin el cual no se revisa el criterio de fuentes y citas).*